

Lattice-based Multi-message Multi-recipient KEM/PKE with Malicious Security

Zeyu Liu¹ Katerina Sotiraki¹ Eran Tromer² Yunhao Wang¹

¹Yale University

²Boston University

Abstract

The efficiency of Public Key Encryption (PKE) and Key Encapsulation Mechanism (KEM), and in particular their large ciphertext size, is a bottleneck in real-world systems. This worsens in post-quantum secure schemes (e.g., lattice-based ones), whose ciphertexts are an order of magnitude larger than prior ones.

The work of Kurosawa (PKC '02) introduced multi-message multi-recipient PKE (mmPKE) to reduce the amortized ciphertext size when sending messages to more than one recipient. This notion naturally extends to multi-message multi-recipient KEM (mmKEM).

In this work, we first show concrete attacks on existing lattice-based mmPKE schemes: Using maliciously-crafted recipient public keys, these attacks completely break semantic security and key privacy, and are inherently undetectable.

We then introduce the first lattice-based mmKEM scheme that maintains full privacy even in the presence of maliciously-generated public keys. Concretely, the ciphertext size of our mmKEM for 100 recipients can be $\sim 8\times$ smaller than naively using Crystals-Kyber. We also show how to extend our mmKEM to mmPKE, achieving a scheme that outperforms all prior lattice-based mmPKE schemes in terms of both security and efficiency. We additionally show a similar efficiency gain when applied to batched random oblivious transfer, and to group oblivious message retrieval.

Our scheme is proven secure under a new Module-LWE variant assumption, Oracle Module-LWE, which can be of its own independent interest. We reduce standard MLWE to this new assumption for some parameter regimes, which also gives intuition on why this assumption holds for the parameter we are interested in (along with additional cryptanalysis).

Furthermore, we show an asymptotically efficient compiler that removes the assumption made in prior works that recipients know their position in the list of intended recipients for every ciphertext.

1 Introduction

Public Key Encryption (PKE) and Key Encapsulation Mechanisms (KEM) are two essential and widely used cryptographic tools. However, in many applications, they are costly compared to other system components (e.g., symmetric-key cryptography), and thus significantly affect overall latency, communication and storage, as well as extra protocol complexity to amortize their cost. One major cost is the ciphertext size of PKE and KEM, made even worse by the ongoing transition to post-quantum schemes whose ciphertexts are larger. For example, the NIST ML-KEM (Kyber) post-quantum KEM [13, 22], based on lattices, has a ciphertext of 800 bytes, compared to 64 bytes

for quantum-insecure ElGamal, based on elliptic curves. These ciphertexts are much larger than other typical system components, such as AES symmetric-key encryption with 16-byte ciphertexts. Thus, PKE/KEM can become a communication bottleneck in large-scale systems, especially with short plaintexts and large recipient groups (e.g., [4, 37]).

Motivated by this, Kurosawa [33] introduced multi-recipient encryption (MRE), where the goal is to send one ciphertext to $T > 1$ recipients such that the total ciphertext size is much smaller than independently sending T ciphertexts. This primitive has been explored in follow-up works [7, 6, 45, 46, 28, 26, 2, 38]. Several variants are proposed; specifically, *multi-recipient PKE (mPKE)* and *multi-recipient KEM (mKEM)* refer to the case of encrypting a single plaintext message to multiple recipients. For encrypting different messages to each recipient, the corresponding notions are *multi-message multi-recipient PKE (mmPKE)* and *multi-message multi-recipient KEM (mmKEM)* [2].

It is known how to achieve mmPKE, and thus all the above notions, from lattice assumptions [38]. However, the security only holds assuming that all recipients' public keys are honestly generated, which cannot be guaranteed in applications where recipients generate their own keys and some of them may be malicious or compromised. Thus, security should be strengthened to hold even when some of the recipients' public keys are maliciously generated. This was achieved for mmKEM and mmPKE under quantum-insecure assumptions (decisional Diffie-Hellman) [45], but in the post-quantum setting, only mKEM (i.e., sending the same plaintext to all recipients) is achieved [28, 26, 2]. Furthermore, no known post-quantum mmPKE scheme is fully secure against maliciously-generated keys. The closest is a construction of mmPKE from mKEM [2], which leaks information (regarding how many recipients share the same plaintexts) and has the same efficiency as the trivial solution (sending T independent ciphertexts) when all the plaintexts are distinct.

Therefore, the following natural question remains open:

Can we build mmKEM and mmPKE that are fully secure against maliciously-generated public keys, based on a post-quantum secure assumption, and are more efficient than the trivial solution?

This paper indeed shows such mmKEM/mmPKE schemes, and analyzes their concrete and asymptotic efficiency. The security of our schemes is proven under a new Module-LWE variant, Oracle Module-LWE. Moreover, we achieve an additional security notion, *key privacy*, needed by some applications; and we remove the need for recipients to somehow learn their positions in the addressed groups.

1.1 Our Contribution

Attacks on existing mmPKE/mmKEM schemes. As mentioned, [38] achieves mmPKE with full privacy assuming all public keys are honestly generated. We show concrete attacks that break both the semantic security and the key-privacy of that scheme, when recipients may provide maliciously-generated public keys. These attacks extend also to the mmKEM variant of the mmPKE in [38], and are inherently *undetectable* (under the same LWE hardness assumptions as the scheme itself).

Efficient key-private lattice-based mmKEM. We construct the first non-trivial lattice-based mmKEM scheme that tolerates maliciously-generated public keys. We present two constructions, where the first one is based on a Module-LWE variant described below with *super-polynomial* modulus-to-noise-ratio, and the second one is based on the same variant of Module-LWE with

polynomial modulus-to-noise-ratio. Our results naturally extend to mmPKE using the generic KEM-DEM paradigm.

Furthermore, we formally define a notion of key-privacy for mmKEM and mmPKE (even in the presence of maliciously-generated public keys), extending key-privacy for PKE introduced in [5]. We then show that our constructions are key-private.

The ciphertext size of our constructions is much smaller than the trivial solution (concatenating independent ciphertexts). Asymptotically, our ciphertext size is $O(T + |\text{ct}_s|)$ instead of $O(T \cdot |\text{ct}_s|)$, where $|\text{ct}_s|$ is the ciphertext size of single-recipient lattice-based KEM.¹ Concretely, for a group of 100 recipients, our ciphertext size is about $10\times$ smaller than Crystals-Kyber.

Oracle Module-LWE. We introduce a new lattice assumption, Oracle Module-LWE (OMLWE), which shares similarities with static Diffie-Hellman with oracle [16, 45] (detailed in Section 6.1). This assumption is used to build our mmKEM, and might be useful in other applications. We additionally provide cryptanalysis for this new assumption, showing that for some parameter regimes, it is broken, while for some parameters, it is at least *as hard as* regular MLWE. With these, we also discuss intuitively why the assumption holds for the parameters of interest in our application.

Positionless correctness of mmKEM/mmPKE. Most prior works [33, 7, 6, 45, 46, 28, 26, 2] assume the recipient to know its position in the group of intended recipients (e.g., “I am the 17th recipient of this ciphertext”) in order to invoke decapsulation/decryption. However, this burdens applications with conveying position information while preserving key privacy. Therefore, we introduce mmKEM/mmPKE with *positionless correctness*, where the recipient does not need to know such information for correct decapsulation/decryption. We additionally show an efficient compiler that compiles any mmKEM/mmPKE with regular correctness to an mmKEM scheme with positionless correctness. In particular, for a group of T recipients, the encapsulation/encryption takes $O(T^{1+\epsilon})$ time (for any $\epsilon > 0$) and the decapsulation/decryption can be done in $O(\text{polylog}(T))$ time.

Evaluations. We carefully choose the parameters for our schemes, achieving 128-bit security and 128-bit correctness guarantee. We choose two different sets of parameters, and show that our scheme improves the ciphertext size by $2\text{-}16\times$ for a group of 10-1000 recipients. The two sets of parameters have their own tradeoffs (one behaves better for smaller groups and one behaves better for larger groups). We include a more detailed discussion on how our scheme behaves in Section 10. We additionally show that we can achieve a similar communication cost saving when applying our mmKEM/mmPKE to batched random oblivious transfer and group oblivious message retrieval, which only require CPA-security.

1.2 Technical Overview

Before going under the hood, we summarize the main techniques used for the contributions above.

Attacks on existing mmPKE. We start with attacks on existing mmPKE schemes. Recall that the main idea of existing lattice-based mmPKE schemes is that every recipient i , with secret key sk_i , shares a matrix $A \in \mathbb{Z}_q^{n \times n}$ and publishes its public key $\text{pk}_i := A\text{sk}_i + E_i \in \mathbb{Z}_{n \times \ell}$. To encrypt, the sender computes $\text{ct}_0 \leftarrow xA + \vec{e}'$, $\text{ct}_i \leftarrow x\text{pk}_i + \vec{e}_i + \frac{q}{p} \cdot \vec{m}_i$ for message $\vec{m} \in \mathbb{Z}_p^\ell$.

We introduce two types of attacks: the single recipient attack and the multi-recipient attack. Suppose that the adversary only targets a single recipient with pk_i , it could simply copy-paste its

¹For mmPKE, this assumes for simplicity that the plaintext size is constant; otherwise there is another inevitable $O(T \cdot |\text{pt}|)$ term, where $|\text{pt}|$ is the plaintext size.

public key $\mathbf{pk}_A := \mathbf{pk}_i$. After receiving the ciphertext, if $\mathbf{ct}_i \approx \mathbf{ct}_A$, then $\vec{m}_i = \vec{m}_A$ and vice versa. This immediately breaks semantic security. Moreover, such attacks could be easily extended to be undetectable under the standard LWE assumption. If the adversary has more power (i.e., can control more recipients), it could use $\text{poly}(\lambda)$ number of $\mathbf{pk}_A$'s to generate a trapdoor and thus recover the randomness from the encryption procedure. The adversary is then able to recover the messages of everyone else in the group.

This, unfortunately, extends to mmKEM as well. Even if $\vec{m}_i$ is uniformly sampled from $\{0,1\}^\ell$ (with $\ell = \lambda/\log(p)$ for λ bits of keys) as for the mmKEM case, we can still launch similar attacks. For example, copying another recipient's public key can break key privacy. For simplicity, assume $p = 2$, then $\mathbf{ct}_A - \mathbf{ct}_i \approx \{0, q/2\}^\ell$ only if $\mathbf{ct}_i$ is the victim's ciphertext (except with small probability). So, an attacker can find out if the intended recipient is the victim by just checking whether $(\mathbf{ct}_A - \mathbf{ct}_i)[j] \approx 0$ or $q/2$ for all $j \in [\ell]$. We discuss these attacks in more details in Section 5.

Patch the scheme with a uniform mask. Our first idea for building a mmKEM is to introduce a fully uniform mask on the ciphertext. In other words, let $\mathbf{ct}_i \leftarrow x\mathbf{pk}_i + \vec{e}_i + \vec{u}$, where $\vec{u} \xleftarrow{\$} \mathbb{Z}_q^\ell$. Then, the encapsulated key is $H(\lfloor p\vec{u}/q \rfloor)$ where H is a random oracle and p is some value $< q$. With this, all the above attacks (and the extended ones discussed in Section 5) do not work anymore. We are not able to prove it secure directly under standard lattice-based assumptions. Thus, we propose a new variant called Oracle Module-LWE (OMLWE), introduced later in this section.

Guaranteeing correctness with super-polynomial modulus. An immediate question that arises from this scheme is how to guarantee correctness. In particular, when decrypting, the recipient *does not* obtain $\vec{u}$, but only an approximation, i.e., $\vec{u}' \approx \vec{u}$. This means that one might have $H(\lfloor p\vec{u}/q \rfloor) \neq H(\lfloor p\vec{u}'/q \rfloor)$ for the random oracle H , making it hard for the sender and the recipient to agree on a key. A simple solution is to assume that there is a superpolynomial gap between p and q , and $|\chi| = \text{poly}(\lambda)$, where χ is the noise distribution (for all the aggregated noise, for simplicity). Then, except for $\text{negl}(\lambda)$ probability, $H(\lfloor p\vec{u}/q \rfloor) = H(\lfloor p\vec{u}'/q \rfloor)$.

Guaranteeing correctness with polynomial modulus. Super-polynomial modulus-to-noise ratio should be avoided to achieve better security and efficiency guarantees. However, when the modulus-to-noise ratio is polynomial, the correctness issue resurfaces. To obtain a protocol in this regime, we first observe that by setting the parameters correctly, $\vec{u}'$ is not arbitrary in $\mathbb{Z}_q$ even with the uniform mask $\vec{u}_i$. Specifically, we set $|\chi| < q/p$ and make sure all the noises are positive. Then, it holds that $\lfloor p \cdot \vec{u}'[i]/q \rfloor \in \{\lfloor p \cdot \vec{u}[i]/q \rfloor, \lfloor p \cdot \vec{u}[i]/q \rfloor + 1\} \subseteq \mathbb{Z}_p$ for all $i \in [\ell]$.

Testing all these possible keys to recover $\vec{u}$ takes 2^ℓ time, which is super-polynomial in the security parameter λ since $p^\ell \geq 2^\lambda$ and $p = \text{poly}(\lambda)$. To fix this, we observe that before rounding the recipient has additional information about $\vec{u}' := \mathbf{ct}_i - \langle \mathbf{ct}_0, \mathbf{sk}_i \rangle$, thus reducing the number of possible keys greatly.

In more detail, assume that $q/p = c \cdot |\chi|$ for some parameter c . Notice that $\vec{u}' = \vec{u} + \vec{e} \bmod q$ and for all i , and $\vec{e}[i] \leq |\chi|$. Thus, if $\vec{u}'[i]$ is greater than a multiple of $q/p = c \cdot |\chi|$ by at least $|\chi|$, then $\vec{u}[i] = \vec{u}'[i] - \vec{e}[i]$ is greater than the exact same multiple of q/p . Hence, $\lfloor p \cdot \vec{u}'[i]/q \rfloor = \lfloor p \cdot \vec{u}[i]/q \rfloor$. Thus, if $\lfloor p \cdot \vec{u}'[i]/q \rfloor \neq \lfloor p \cdot \vec{u}[i]/q \rfloor$, then $\vec{e}'[i] \leq |\chi|$.

Now, suppose there are ℓ' elements in $\vec{u}'$ that are greater than a multiple of q/p by at most $|\chi|$, then this means that we only need to check these ℓ' elements, leading to $2^{\ell'}$ possible keys. Note that for mmKEM, $\ell = O(\lambda)$ (since $p \geq 2$ and $p^\ell = 2^\lambda$), and by setting $c = \lambda$, we have $\ell' \leq \log \log(\lambda)$ except for $\text{negl}(\lambda)$ probability (see Section 6.3 for details). This means that instead of testing 2^ℓ possible keys, it suffices to test $2^{\log \log(\lambda)}$ keys, which only takes $\text{polylog}(\lambda)$ time.

The only missing piece of the puzzle is how one can test each key: in each ciphertext, the sender

can additionally attach $G(\lfloor p \cdot \vec{u}/q \rfloor)$, for some random oracle $G \neq H$. The recipient can then use G for verification.

Oracle Module-LWE. With correctness figured out, we briefly discuss our new assumption, Oracle Module-LWE. It is similar to Staitc Diffie-Hellman with Oracle (see Section 6.1): given $w_A = \text{poly}(\lambda)$ number of MLWE samples masked as above (together with the random oracle output on the mask), together with w_1 regular MLWE samples, the adversary is asked to distinguish between w_2 honestly generated samples masked as above and elements chosen uniformly at random. We provide additional cryptanalysis on this new assumption in more detail in Section 7. Specifically, we show some parameter regimes where the assumption is broken, and some parameter regimes where it is at least as hard as standard MLWE.

Positionless correctness. Lastly, we discuss a new property we introduce for mmKEM/mmPKE: positionless correctness. Essentially, it means that the recipient inside a group of T recipients does not need to know its position to correctly decrypt the ciphertext. A natural solution is to let the recipient trial decrypt every ciphertext and find the one addressed to it, but this takes $\Omega(T)$ time. We provide an asymptotically efficient solution: the sender first interpolates a function $f(\text{pk}) = \text{ct}$ that maps a public key to a ciphertext. Interpolating such a function takes $\tilde{O}_\lambda(T)$ time. Then, it preprocesses the function using techniques in [29], which also takes $\tilde{O}_\lambda(T)$ time. Lastly, the sender uses the function as a ciphertext. The recipient can then evaluate this function with its public key, which takes only $\text{poly}(\lambda, \log(T))$ time [29], and then decrypt the resulting ciphertext.

2 Related Works

2.1 Prior Works in mmPKE/mmKEM

Lattice-based mmPKE. As shown in Table 1, our construction of mmPKE is the only existing lattice-based scheme that achieves full privacy *and* has ciphertext size $O(T + |\text{ct}_s|)$ (see Table 1).

The constructions of [28, 26, 2] do not achieve full privacy since they would leak whether two consecutive recipients share the same plaintext. At a high level, they first construct an mKEM where all recipients share the same encapsulated key and the plaintext is encrypted under this same encapsulated key (using a data encapsulation mechanism). They then use mKEM as a building block to construct an mmKEM. However, if any 2 consecutive recipients among T share the same plaintext, their mmKEM sends a single key to both recipients. Similarly, if all T recipients share the same plaintext, they share the same key with that plaintext encrypted under this key. Assume all plaintexts are lexicographically sorted and let t be the number of plaintexts distinct from their left neighbors. The ciphertext size is essentially $t \cdot |\text{ct}_s|$. It is easy to see that t by itself already leaks a lot of information.

On the other hand, [38] achieves mmPKE (under LWE, but easily extendable to module-LWE), with ciphertext size asymptotically comparable to ours. However, it does not tolerate malicious public keys, i.e., all public keys should be honestly generated. Therefore, it is only useful in specific applications.

One can also consider two different notions of correctness, as defined in Section 4. Roughly speaking, a scheme is *correct* if decryption/decapsulation works when the recipient knows its position in the group (i.e., it knows that it is recipient $i \in [T]$ among the targeted T recipients). *Positionless correctness* means that decryption/decapsulation works even when this information is not available to the recipient.

	Ciphertext Size	Privacy Guarantee	Correctness Guarantee	Model Assumption	CCA Security
Naive PKE	$O(T \cdot \text{ct}_s)$	Full	Regular	N/A	Via FO transformation
[28, 26]	$O(T + t \cdot \text{ct}_s)$	With leakage			N/A
[38]	$O(T + \text{ct}_s)$	No malicious PK	$\Omega(T)$ Positionless	ROM	via zk-SNARK
This work	$O(T + \text{ct}_s)$	Full	Positionless		

Table 1: Comparisons with prior works in lattice-based mmPKE. T is the number of the intended recipients. $|\text{ct}_s|$ is the ciphertext size of a plain lattice-based PKE scheme ([40, 13]). $1 \leq t \leq T$ is the number of distinct plaintexts in a vector of plaintexts (see Section 2.1 for details). Preferred characteristics are marked in green. We assume the plaintext size to be constant. See a more detailed correctness guarantee comparison in Section 2.1.

It is clear that regular correctness can be a troublesome assumption in practice (especially if the scheme needs to be key-private). Looking ahead, we present an asymptotically efficient compiler (Section 8) to compile a scheme with regular correctness to positionless correctness. Note that our compiler is for mmKEM instead of mmPKE, but it is applicable to prior works [33, 7, 6, 45, 46, 28, 26, 2], and in general, easily extendable to the corresponding mmPKE scheme. While [38] also achieves positionless correctness, their decryption time is $\Omega(T)$ thus inefficient, unlike ours, which only requires $\tilde{O}(1)$ (see Section 8 for more details). Our constructions focus on CPA security, but for completeness, we discuss CCA security in Section 6.4.

Lattice-based mmKEM. No prior work presents a lattice-based mmKEM scheme. All prior works either achieve mKEM (i.e., all recipients share the same encapsulated key) instead of mmKEM [28, 26, 2] or directly achieve mmPKE without mmKEM [38], which can be adapted to mmKEM but does not guarantee security under malicious keys.

A recent concurrent and independent work [50] also achieves lattice-based mmKEM. See Appendix A for details.

Other mmPKE/mmKEM. The notion of mmPKE (or MRE) was first introduced in [33] and later improved in [7, 6, 45, 46], all of which are based on non-lattice-based assumptions, such as DDH. Among them, the only construction that achieves the same privacy guarantee and has the same asymptotic ciphertext size as ours is [46]. They construct an ElGamal version of mmPKE, which allows maliciously-generated public keys. While they do not allow corruption in their definition, their construction naturally allows it. Compared to them, the main advantage of our construction is the plausibly post-quantum security. It can similarly achieve positionless correctness using our compiler.

2.2 Dual Encryption

Dual Encryption was introduced in [21] and further studied in [17, 52, 9]. Essentially, it is an mPKE intended for two recipients (i.e., send the same plaintext to two recipients). [17] enhanced dual encryption by introducing the soundness notion, such that a ciphertext must be decrypted into the same plaintext by the two recipients, which is the converse of mmPKE allowing every recipient to receive a different message.

3 Preliminaries

3.1 Notation

Let N be a power of two. Let $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ denote the $2N$ -th cyclotomic ring where N is a power-of-two, and $\mathcal{R}_Q = \mathcal{R}/Q\mathcal{R}$ for some $Q \in \mathbb{Z}$. Let $[n]$ denote the set $\{1, \dots, n\}$. Let $\|\vec{x}\|_\ell := (\sum_{i \in [n]} \vec{x}[i]^\ell)^{1/\ell}$ denote the ℓ -norm for vector $\vec{x}$. For simplicity, we use $|x|$ to represent $\|x\|_\infty$. If $x \in \mathcal{R}$, let $\|x\|_\ell$ denote the ℓ -norm of the coefficient vector of x , and let $x[i]$ denote the i -th coefficient of x . For $x \in \mathcal{R}, \vec{y} \in \mathbb{Z}^\ell$ where $\mathcal{R}$ has ring dimension N and $0 < \ell \leq N$, we define $x + \vec{y}$ to be: let $\vec{x} \in \mathbb{Z}^\ell$ denote the vector of the first ℓ coefficients of x , and $x + \vec{y} := \vec{x} + \vec{y} \in \mathbb{Z}^\ell$ (naturally extends to $\mathbb{Z}_q^\ell$ for any $q > 0$).

Let $\mathcal{D}$ denote an arbitrary distribution. Let $x \xleftarrow{\$} P$ denote a uniformly random sample from space P , $x \leftarrow \mathcal{D}$ denotes a uniform sample from distribution $\mathcal{D}$. Let $\mathcal{D}_\beta$ denote a distribution with norm bound β , s.t. $\Pr_{x \leftarrow \mathcal{D}_\beta}[|x| > \beta] \leq \text{negl}(\lambda)$ for security parameter λ . Let $\mathcal{D}_{+, \beta}$ be $\mathcal{D}_\beta$ shifted by β (i.e., let $x \leftarrow \mathcal{D}_{+, \beta}$, then it is equivalent to first sample $x' \leftarrow \mathcal{D}_\beta$ and let $x \leftarrow x' + \beta$).

Let χ_σ be a discrete Gaussian distribution with mean being 0 and standard deviation being σ , and let $|\chi_\sigma|$ be the norm bound of this distribution (i.e., $\Pr_{x \leftarrow \chi_\sigma}[|x| > |\chi_\sigma|] \leq \text{negl}(\lambda)$). Then, let $\chi_{+, \sigma}$ be the Gaussian distribution with mean $|\chi_\sigma|$ and standard derivation σ , s.t., $\Pr_{x \xleftarrow{\$} \chi_{+, \sigma}}[x \in [0, 2|\chi_\sigma| + 1]] \geq 1 - \text{negl}(\lambda)$. In other words, let $x \leftarrow \chi_{+, \sigma}$, then it is equivalent to first sample $x' \leftarrow \chi_\sigma$ and then $x \leftarrow x' + |\chi_\sigma|$. Let $\mathcal{D}^\ell$ represent the joint distribution $\mathcal{D} \times \mathcal{D} \times \dots \times \mathcal{D}$ (e.g. if $\vec{x} \leftarrow \mathcal{D} \in \mathbb{Z}_q^n$, then $X \leftarrow \mathcal{D}^\ell \in \mathbb{Z}_q^{n \times \ell}$).

3.2 Hard Problem

Definition 3.1 (Decisional module learning with error problem [40]). Let $n, N, q, \mathcal{D}, \chi$ be parameters dependent on λ and N being a power of two. Let $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$. The ring learning with error (Module-LWE or MLWE) problem $\text{MLWE}_{n, N, q, \mathcal{D}, \chi}$ is the following: for any $w = \text{poly}(\lambda)$, distinguish $(A, A \cdot \vec{s} + \vec{e})$ and $(A, \vec{b})$ (with noticeable advantage), where $A \xleftarrow{\$} \mathcal{R}_q^{w \times n}, \vec{s} \leftarrow \mathcal{D}^n, \vec{e} \leftarrow \chi^w$ and $\vec{b} \xleftarrow{\$} \mathcal{R}_q^w$.

LWE and RLWE. Standard LWE is simply Module-LWE with $N = 1$ (i.e., ring dimension) and standard RLWE (or RingLWE) is simply Module-LWE with $n = 1$ (i.e., length of the secret).

3.3 Data Encapsulation Mechanism

Let $\mathcal{K}$ be the key space, and $\mathcal{M}$ be the message space. A *Data Encapsulation Mechanism (DEM)* scheme, i.e., symmetric key-encryption as defined in [47], contains two PPT algorithms: the encryption algorithm $\text{Enc}(\lambda, k, m) \rightarrow c$, given a security parameter λ , a key $k \in \mathcal{K}$, and a message $m \in \mathcal{M}$, outputs a ciphertext c ; and the (deterministic) decryption algorithm $\text{Dec}(k, c) \rightarrow m$, given a key k and a ciphertext c , outputs a message m .

Correctness and security are defined as follows.

Definition 3.2. (Correctness) Let $k \xleftarrow{\$} \mathcal{K}$, for any $m \in \mathcal{M}, \lambda > 0$, let $c \leftarrow \text{Enc}(\lambda, k, m)$, and $m' \leftarrow \text{Dec}(k, c)$, it holds that $\Pr[m = m'] \geq 1 - \text{negl}(\lambda)$.

Definition 3.3. (Security) For $\text{ATK} \in \{\text{CPA}, \text{CCA}\}$, a DEM scheme is *ATK-secure* if for $k \xleftarrow{\$} \mathcal{K}$, for any PPT adversary $\mathcal{A} := (\mathcal{A}_1, \mathcal{A}_2)$, and $\lambda > 0$, letting $(m_0, m_1, \text{st}) \xleftarrow{\$} \mathcal{A}_1^{\text{Enc}_k, \text{Dec}_k}(\lambda)$, $b \xleftarrow{\$} \{0, 1\}$, $c \leftarrow \text{Enc}(\lambda, k, m_b)$, $b' \leftarrow \mathcal{A}_2^{\text{Enc}_k, \text{Dec}_k}(\text{st}, c)$; it holds that $\Pr[b = b'] \leq 1/2 + \text{negl}(\lambda)$. The oracle $\text{Dec}_k(c')$ returns $\text{Dec}(k, c')$ if $c \neq c'$ and $\text{ATK} = \text{CCA}$, otherwise it returns $\perp$.

3.4 Multi-variate Polynomials

We recall two results introduced for multi-variate polynomials in [35] and [29]. These are used later (in Section 8) to build an efficient compiler for mmKEM schemes to achieve positionless correctness. Essentially, the first result states that one can efficiently interpolate an m -variate polynomial with individual degree d (i.e., each variable has at most degree d), formally stated as follows.

Lemma 3.4. (Polynomial Interpolation [35, Lemma 2.2]). *Let $\mathbb{F}_q$ be a prime field, and let $m \in \mathbb{N}$ be an integer. Let $\{y_{x_1, \dots, x_m} \in \mathbb{F}_q\}_{x_1, \dots, x_m \in S \subseteq \mathbb{F}_q^m, |S|=d^m}$ be any set of d^m values. There exists an algorithm that runs in quasi-linear time $O(d^m \cdot m \cdot \text{polylog}(d))$ and recovers the coefficients of a polynomial $f(X_1, \dots, X_m) \in \mathbb{F}_q[X_1, \dots, X_m]$ with individual degree $< d$ in each variable such that $f(x_1, \dots, x_m) = y_{x_1, \dots, x_m}$ for all $(x_1, \dots, x_m) \in \mathbb{F}_q^m$.*

The second result states that one can preprocess the m -variate polynomial f with individual degree d efficiently into $\tilde{f}$ such that $\tilde{f}(\cdot) = f(\cdot)$ for any input and evaluating $\tilde{f}$ is much faster than evaluating f , formally as follows (we use the version in [35, Theorem 2.1], adapted from [29, Theorem 5.1]).

Theorem 3.5. (Polynomial Preprocessing [29, Theorem 5.1]). *For a m -variate polynomial $P : \mathbb{F}_q^m \rightarrow \mathbb{F}_q$ with individual degree $\leq d$, over some finite field $\mathbb{F}_q$, then, there exists an algorithm PolyPreProcess such that (1) the runtime of $\text{PolyPreProcess}(P)$ is $d^m \cdot \text{poly}(d, m, \log(q))$; (2) let $\tilde{P} \leftarrow \text{PolyPreProcess}(P)$, for any $\vec{x} \in \mathbb{F}_q^m$, $\tilde{P}(\vec{x}) = P(\vec{x})$ and that the runtime of $\tilde{P}(\vec{x})$ is $\text{poly}(d, m, \log(q))$.*

3.5 Zero-knowledge SNARKs

In this section, we briefly recall the definition of zero-knowledge succinct non-interactive argument of knowledge (zk-SNARK) introduced in [11].

NP-relation. NP-relation is defined to be a set $\mathcal{R}$ of instance-witness pairs (y, w) , where $y = (M, x, t)$, $|w| \leq \text{poly}(|x|)$, and M is a Turing machine such that M accepts (x, w) after at most $\text{poly}(|x|)$ steps.

zk-SNARK. A zk-SNARK construction has the following PPT algorithms.

- $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$: takes a security parameter λ and outputs a public parameter pp
- $\pi \leftarrow \text{Prove}(\text{pp}, y, w)$: takes a public parameter, an instance y and witness w , outputs a proof π .
- $b \leftarrow \text{Verify}(\text{pp}, y, \pi)$: takes a public parameter pp , an instance y , and a proof π , outputs a bit b .

A zk-SNARK construction holds the following properties.

IND-ATK $_{\text{mmKEM}, P}^{\mathcal{A}=(\mathcal{A}_1, \mathcal{A}_2)}(1^\lambda)$, ATK $\in \{\text{CPA}, \text{CCA}\}$	KP-ATK $_{\text{mmKEM}, P}^{\mathcal{A}=(\mathcal{A}_1, \mathcal{A}_2)}(1^\lambda)$, ATK $\in \{\text{CPA}, \text{CCA}\}$
<pre> 1 : $b \xleftarrow{\\$} \{0, 1\}$ 2 : $\text{pp} \leftarrow \text{mmKEM.GenParams}(1^\lambda)$ 3 : for $i \in [P]$: $(\text{pk}_i, \text{sk}_i) \leftarrow \text{mmKEM.KeyGen}(\text{pp})$ 4 : $\text{ct} \leftarrow \perp$ 5 : $(\vec{\text{pk}}^*, \text{st}) \leftarrow \mathcal{A}_1^{\text{Decaps}^{\text{ATK}}}(\text{pp}, \text{pk}_1, \dots, \text{pk}_P)$ 6 : Let $T := \vec{\text{pk}}^*$ 7 : $(\text{ct}, \vec{k}_0) \leftarrow \text{mmKEM.Encaps}(\text{pp}, \vec{\text{pk}}^*)$ 8 : $\vec{k}_1 \xleftarrow{\\$} \mathcal{K}^T$ 9 : $\forall j \in [T]$, if $\vec{\text{pk}}^*[j] \notin (\text{pk}_i)_{i \in [P]}$ 10 : then $\vec{k}_1[j] \leftarrow \vec{k}_0[j]$ 11 : $b' \leftarrow \mathcal{A}_2^{\text{Decaps}^{\text{ATK}}}(\text{st}, \text{ct}, \vec{k}_b)$ 12 : return $b = b'$ </pre>	<pre> 1 : $b \xleftarrow{\\$} \{0, 1\}$ 2 : $\text{pp} \leftarrow \text{mmKEM.GenParams}(1^\lambda)$ 3 : for $i \in [P]$: $(\text{pk}_i, \text{sk}_i) \leftarrow \text{mmKEM.KeyGen}(\text{pp})$ 4 : $\text{ct} \leftarrow \perp$ 5 : $(\vec{\text{pk}}_0^*, \vec{\text{pk}}_1^*, \text{st}) \leftarrow \mathcal{A}_1^{\text{Decaps}^{\text{ATK}}}(\text{pp}, \text{pk}_1, \dots, \text{pk}_P)$ 6 : Req $T := \vec{\text{pk}}_0^* = \vec{\text{pk}}_1^*$ 7 : $(\text{ct}, \vec{k}) \leftarrow \text{mmKEM.Encaps}(\text{pp}, \vec{\text{pk}}_b^*)$ 8 : $b' \leftarrow \mathcal{A}_2^{\text{Decaps}^{\text{ATK}}}(\text{st}, \text{ct}, \vec{k})$ 9 : Req $\forall j \in [T]$: $\vec{\text{pk}}_0^*[j] \notin (\text{pk}_i)_{i \in [P]} \implies \vec{\text{pk}}_0^*[j] = \vec{\text{pk}}_1^*[j]$ 10 : return $b = b'$ </pre> <pre> Oracle Decaps$^{\text{ATK}}(i, j, c)$ 1 : if ATK = CPA or $c = \text{ct}$ or $i \notin [P]$: return $\perp$ 2 : else : return $\text{mmKEM.Decaps}(\text{pp}, \text{sk}_i, c, j)$ </pre>

Figure 1: mmKEM security games. $P = \text{poly}(\lambda)$ is the total number of parties in the system, while T is the number of parties per ciphertext.

- (Correctness) For any $(y, w) \in R$, let $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$, $\pi \leftarrow \text{Prove}(\text{pp}, y, w)$, it holds that $\Pr[\text{Verify}(\text{pp}, y, \pi) = 1] \geq 1 - \text{negl}(\lambda)$.
- (Succinctness) For the same quantifier as correctness, $|\pi|$ and the runtime of Prove are both bounded by $\text{poly}(\lambda) \cdot \text{polylog}(|x|)$.
- (Knowledge) For any poly-size prover P , there exists a poly-size extractor $\mathcal{E}_P$ such that for all large enough $\lambda > 0$, and all auxiliary inputs $z \in \{0, 1\}^{\text{poly}(\lambda)}$, let $\text{pp} \leftarrow \text{GenParam}(1^\lambda)$, $(y, \pi) \leftarrow P(\text{pp}, z)$, if $\text{Verify}(\text{pp}, y, \pi) = 1$, let $w' \leftarrow \mathcal{E}_P(z, \text{pp})$, then $\Pr[w' \notin \mathcal{R}(y)] \leq \text{negl}(\lambda)$.

4 Multi-message Multi-recipient Key Encapsulation Mechanism

A *Multi-message Multi-recipient Key Encapsulation Mechanism (mmKEM)* consists of the following four PPT algorithms (taken from [45]).²

- $\text{pp} \leftarrow \text{GenParam}(1^\lambda, \text{aux})$: takes a security parameter λ and some auxiliary input aux ; outputs a public parameter pp .
- $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(\text{pp})$: takes a public parameter pp ; outputs a secret and public key pair (sk, pk) .

²As in prior works [28, 26], we focus on the so-called *decomposable* mmKEM: i.e., the Encaps algorithm can be decomposed into a key-independent algorithm $\text{Encaps}^{\text{ind}}$ and a key-dependent algorithm $\text{Encaps}^{\text{dep}}$, combined as in Algorithm 1. To our knowledge, all the existing mmKEM/mmPKE are decomposable [33, 7, 6, 45, 46, 28, 26].

- $(\text{ct} = (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_T), \vec{k} = (k_1, \dots, k_T)) \leftarrow \text{Encaps}(\text{pp}, \vec{\text{pk}} = (\text{pk}_1, \dots, \text{pk}_T); r_0, r_1, \dots, r_T)$: takes the public parameter pp , a list of T recipient public keys $\vec{\text{pk}}$, and randomness $r_0, \dots, r_T$; outputs a key k_i , and a ciphertext $(\text{ct}_0, \text{ct}_i)$ encapsulating k_i , for every recipient $i \in [T]$. This is an algorithm defined via black-box use of algorithms $\text{Encaps}^{\text{ind}}$, $\text{Encaps}^{\text{dep}}$ below. Algorithm 1 shows how all prior works [33, 7, 6, 45, 46, 28, 26, 2] realize it, which is also used in our main algorithms Algorithms 2 and 3. Note that in Algorithm 5, we offer an alternative that achieves stronger correctness.
 - $\text{ct}_0 \leftarrow \text{Encaps}^{\text{ind}}(\text{pp}; r_0)$: takes a public parameter pp with some randomness r_0 , and outputs a public-key-independent ciphertext component ct_0 .
 - $(\text{ct}_i, k_i) \leftarrow \text{Encaps}^{\text{dep}}(\text{pp}, \text{pk}_i; r_0, r_i)$: takes a public parameter pp and a public key pk_i ($i \in [T]$) with some randomness r_0, r_i ; outputs a ciphertext component ct_i addressed to pk_i , and an encapsulated key k_i .
- $k \leftarrow \text{Decaps}(\text{pp}, \text{sk}, \text{ct}, i, \text{aux})$: takes a public parameter pp , a secret key sk , a ciphertext $\text{ct} = (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_T)$, a position index $i \in [T]$, and some auxiliary input aux ; outputs a decapsulated key k .

Algorithm 1 Encaps for mmKEM

```

1: procedure mmKEM.Encaps( $\text{pp}, \vec{\text{pk}} = (\text{pk}_1, \dots, \text{pk}_n); r_0, r_1, \dots, r_n$ )
2:    $\text{ct}_0 \leftarrow \text{Encaps}^{\text{ind}}(\text{pp}; r_0)$ 
3:   for  $i \in [n]$  do
4:      $(\text{ct}_i, k_i) \leftarrow \text{Encaps}^{\text{dep}}(\text{pp}, \text{pk}_i; r_0, r_i)$ 
5:   return  $(\text{ct} := (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_n), \vec{k} = (k_i)_{i \in [n]})$ 

```

Two correctness definitions. The standard notion of mmKEM correctness [33, 7, 6, 45, 46, 28, 26, 2] requires that a recipient $i \in [T]$ can correctly obtain the encapsulated key k_i , assuming it knows its own position i in the list of recipients.

Definition 4.1 (mmKEM (regular) correctness). Let $\text{pp} \leftarrow \text{GenParam}(1^\lambda, \text{aux})$; for any $T = \text{poly}(\lambda)$, let $(\text{sk}_i, \text{pk}_i) \leftarrow \text{KeyGen}(\text{pp})$ for all $i \in [T]$; let $(\text{ct}, \vec{k}) \leftarrow \text{Encaps}(\text{pp}, (\text{pk}_i)_{i \in [T]})$; for all $i \in [T]$, let $k_i \leftarrow \text{Decaps}(\text{pp}, \text{sk}_i, \text{ct}, i, \text{aux})$, it holds that $\Pr[k_i = \vec{k}[i]] \geq 1 - \text{negl}(\lambda)$.

To remove the assumption, we define a positionless version of correctness, where recipients do not need to know their position index i in the recipients list in order to recover the encapsulated key k_i addressed to their public key (formally, they can pass $i = \perp$ when invoking Decaps). This can benefit applications such as key-private multi-recipient encryption (MRE) schemes, as it avoids the need for an agreement of recipients' order between the sender and the recipients, which is hard to achieve in practice.

Definition 4.2 (mmKEM positionless correctness). Let $\text{pp} \leftarrow \text{GenParam}(1^\lambda, \text{aux})$; for any $T = \text{poly}(\lambda)$, let $(\text{sk}_i, \text{pk}_i) \leftarrow \text{KeyGen}(\text{pp})$ for all $i \in [T]$ and $(\text{ct}, \vec{k}) \leftarrow \text{Encaps}(\text{pp}, (\text{pk}_i)_{i \in [T]})$. For all $i \in [T]$, let $k_i \leftarrow \text{Decaps}(\text{pp}, \text{sk}_i, \text{ct}, \perp, \text{aux})$, it holds that $\Pr[k_i = \vec{k}[i]] \geq 1 - \text{negl}(\lambda)$.

We achieve this stronger correctness property and, looking ahead, show a general compiler to achieve it with high asymptotic efficiency ($\text{polylog}(T)$ time for decapsulation).

Security. Intuitively, we want the adversary, given the power to maliciously craft some of the public keys in $\vec{pk}$ passed to **Encaps**, to learn nothing about the encapsulated keys k_i of honest parties i . We define the indistinguishability security notions IND-CPA and IND-CCA as follows, using the security games in Fig. 1 (left side).

Definition 4.3 (mmKEM security). An mmKEM scheme is IND-ATK secure, where $ATK \in \{CPA, CCA\}$, if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2), P > 0, \lambda > 0$, it holds that $|\Pr[\text{IND-ATK}_{\text{mmKEM}, P}^*(1^\lambda)] - \frac{1}{2}| \leq \text{negl}(\lambda)$.

In more detail, in the IND-ATK game (Fig. 1), the adversary $\mathcal{A}_1$ first takes all the honestly-generated public keys $(pk_1, \dots, pk_P)$ and outputs a vector of public keys $\vec{pk}^*$ (which can be either maliciously generated or one of the P honest public keys), which is the intended group of recipients (line 5). Then, the challenger generates a ciphertext encapsulating the keys $\vec{k}_0$ (line 7) for this intended group, and sample another vector of keys $\vec{k}_1$ (line 8) to be distinguished. Note that if a public key in $\vec{pk}^*$ is maliciously crafted, the corresponding key in $\vec{k}_0$ and $\vec{k}_1$ should be identical to avoid a trivial win: in lines 9-10, the challenger overwrites $\vec{k}_1$ according to $\vec{k}_0$ for such public keys.

Key privacy. We formalize key privacy KP-ATK in Fig. 1 (right side), which requires that if a ciphertext is generated with respect to one of two adversarially chosen public keys, the adversary should not tell which public key(s) is used to generate that ciphertext. Similarly to IND-ATK, we want key privacy (against CPA or CCA) to hold even with maliciously crafted keys.

Definition 4.4 (mmKEM key privacy). An mmKEM scheme is KP-ATK-secure, where $ATK \in \{CPA, CCA\}$, if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2), P > 0, \lambda > 0$, and for $ATK \in \{CPA, CCA\}$, it holds that $|\Pr[\text{KP-ATK}_{\text{mmKEM}, P}^*(1^\lambda)] - \frac{1}{2}| \leq \text{negl}(\lambda)$.

To elaborate on KP-ATK defined in Fig. 1, the adversary $\mathcal{A}_1$ first outputs two vectors of public keys $\vec{pk}_0^*$ and $\vec{pk}_1^*$ after taking all honestly-generated public keys $(pk_1, \dots, pk_N)$ (line 5). The challenger uses one of the vectors to generate a ciphertext **ct** encapsulating a vector of keys $\vec{k}$ (line 7). Note that to avoid trivial attacks, if the i -th location of $\vec{pk}_0^*$ is generated by the adversary, then the i -th location of $\vec{pk}_1^*$ should also use the same public key (lines 9-10).

Definition of mmPKE. mmPKE can be defined in almost the same way as mmKEM, except that it encrypts an arbitrary message instead of encapsulating a key. Due to space, we defer it to Section 4.1.

4.1 Multi-message Multi-recipient Public Key Encryption

A *Multi-message Multi-recipient Public Key Encryption (mmPKE)* scheme consists of four PPT algorithms:

- $pp \leftarrow \text{GenParam}(1^\lambda, \text{aux})$: takes a security parameter λ and some auxiliary input aux ; outputs a public parameter pp .
- $(sk, pk) \leftarrow \text{KeyGen}(pp)$: takes a public parameter pp ; outputs a secret and public key pair (sk, pk) .
- $ct \leftarrow \text{Enc}(pp, \vec{pk} = (pk_1, \dots, pk_T), \vec{m} = (m_1, \dots, m_T))$: takes a public parameter pp , a vector of T public keys, and a vector of T messages; outputs a ciphertext ct .

- $m \leftarrow \text{Dec}(\text{pp}, \text{sk}, \text{ct}, i, \text{aux})$: takes a public parameter pp , a secret key sk , a ciphertext ct , a position index $i \in [T]$, and some auxiliary input aux ; outputs a plaintext m .

Similarly, we define two types of correctness: regular correctness (i.e., the decryption procedure works as needed if i is correctly given) and positionless correctness property (i.e., the decryption protocol works even without the correct i). The semantic security and key privacy are also defined in the way similar to the ones in Section 4.

Definition 4.5 (mmPKE (regular) correctness). Let $\text{pp} \leftarrow \text{GenParam}(1^\lambda, \text{aux})$; for any $T = \text{poly}(\lambda)$, let $(\text{sk}_i, \text{pk}_i) \leftarrow \text{KeyGen}(\text{pp})$ for all $i \in [T]$; for any $m_1, \dots, m_T \in \mathcal{M}$, let $\text{ct} \leftarrow \text{Enc}(\text{pp}, (\text{pk}_i)_{i \in [T]}, (m_i)_{i \in [T]})$; for all $i \in [T]$, let $m'_i \leftarrow \text{Dec}(\text{pp}, \text{sk}_i, \text{ct}, i, \text{aux})$, it holds that $\Pr[m'_i = m_i] \geq 1 - \text{negl}(\lambda)$.

Definition 4.6 (mmPKE positionless correctness). Let $\text{pp} \leftarrow \text{GenParam}(1^\lambda, \text{aux})$; for any $T = \text{poly}(\lambda)$, let $(\text{sk}_i, \text{pk}_i) \leftarrow \text{KeyGen}(\text{pp})$ for all $i \in [T]$; for any $m_1, \dots, m_T \in \mathcal{M}$, let $\text{ct} \leftarrow \text{Enc}(\text{pp}, (\text{pk}_i)_{i \in [T]}, (m_i)_{i \in [T]})$; for all $i \in [T]$, let $m'_i \leftarrow \text{Dec}(\text{pp}, \text{sk}_i, \text{ct}, \perp, \text{aux})$, it holds that $\Pr[m'_i = m_i] \geq 1 - \text{negl}(\lambda)$.

Definition 4.7 (mmPKE security). An mmKEM scheme is IND-ATK-secure, where $\text{ATK} \in \{\text{CPA}, \text{CCA}\}$, if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2), P > 0, \lambda > 0$, it holds that $|\Pr[\text{IND-ATK}_{\text{mmPKE}, P}^*(1^\lambda)] - \frac{1}{2}| \leq \text{negl}(\lambda)$ (Fig. 2 left half).

Definition 4.8 (mmPKE key privacy). An mmKEM scheme is KP-ATK-secure, where $\text{ATK} \in \{\text{CPA}, \text{CCA}\}$, if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2), P > 0, \lambda > 0$, it holds that $|\Pr[\text{KP-ATK}_{\text{mmPKE}, P}^*(1^\lambda)] - \frac{1}{2}| \leq \text{negl}(\lambda)$ (Fig. 2 right half).

5 Attacks on Existing Lattice-based mmPKE and mmKEM

This section discusses the attacks on the only existing lattice-based mmPKE scheme that achieves asymptotic savings with full privacy [38]. Specifically, this construction achieves CPA security and key privacy *if* the public keys are all honestly generated (see [38, Fig. 7]). This section then examines the potential attacks when the public keys are maliciously generated. These attacks completely break the IND-CPA security and key-privacy, and some variants are inherently undetectable (under the same LWE hardness assumption as the scheme itself). Later, we discuss why the other schemes [28, 26, 2] circumvent the attacks by leaking certain information about the plaintexts and by having extra costs.

Existing lattice-based mmPKE schemes. For simplicity, we illustrate the attacks on a simplified version of the mmPKE scheme in [38, Sec 7.3] and its LWE-based encryption. The attack trivially generalizes to the full [38] scheme (including its multi-bit messages and optimizations), as well as its Ring-LWE and Module-LWE variants. The relevant scheme in [38] essentially views existing lattice-based encryption schemes, e.g., [40], as an mmPKE scheme, as follows.

A recipient i samples its secret key $\text{sk}_i \leftarrow \mathcal{D} \in \mathbb{Z}_q^n$, where $\mathcal{D}$ is some distribution, and the noise $\vec{e}_i$ from some Gaussian distribution. The public key is then computed as $\text{pk}_i := A \times \text{sk}_i + \vec{e}_i \in \mathbb{Z}_q^{n \times 1}$, where $A \xleftarrow{\$} \mathbb{Z}_q^{n \times n}$ is shared by all the recipients. To encrypt bits $m_1, \dots, m_T$ to the recipients with public keys $\text{pk}_1, \dots, \text{pk}_T$, the encryption procedure samples $\vec{x} \leftarrow \mathcal{D}$, and outputs $\text{ct} = (\text{ct}_0 := \vec{x}A + \vec{e}_a, \{\text{ct}_i := \vec{x}\text{pk}_i + m_i \cdot \lfloor \frac{q}{2} \rfloor + e_{b,i}\}_{i \in [T]})$ where $\vec{e}_a, e_{b,i}$ are noise vectors sampled from the same Gaussian distribution as $\vec{e}_i$ during key generation above.

IND-ATK $_{\text{mmPKE},P}^{A=(\mathcal{A}_1,\mathcal{A}_2)}(1^\lambda)$	KP-ATK $_{\text{mmPKE},P}^{A=(\mathcal{A}_1,\mathcal{A}_2)}(1^\lambda), \text{ ATK} \in \{\text{CPA}, \text{CCA}\}$
1 : $b \xleftarrow{\$} \{0, 1\}$	1 : $b \xleftarrow{\$} \{0, 1\}$
2 : $\text{pp} \leftarrow \text{mmPKE.GenParams}(1^\lambda)$	2 : $\text{pp} \leftarrow \text{mmPKE.GenParams}(1^\lambda)$
3 : for $i \in [P] : (\text{pk}_i, \text{sk}_i) \leftarrow \text{mmPKE.KeyGen}(\text{pp})$	3 : for $i \in [P] : (\text{pk}_i, \text{sk}_i) \leftarrow \text{mmPKE.KeyGen}(\text{pp})$
4 : $\text{ct} \leftarrow \perp$	4 : $\text{ct} \leftarrow \perp$
5 : $(\vec{\text{pk}}^*, \vec{m}_0, \vec{m}_1, \text{st}) \leftarrow \mathcal{A}_1^{\text{Dec}^{\text{ATK}}}(\text{pp}, \text{pk}_1, \dots, \text{pk}_P)$	5 : $(\vec{\text{pk}}_0^*, \vec{\text{pk}}_1^*, \vec{m}, \text{st}) \leftarrow \mathcal{A}_1^{\text{Dec}^{\text{ATK}}}(\text{pp}, \text{pk}_1, \dots, \text{pk}_P)$
6 : Req $T := \vec{\text{pk}}^* = \vec{m}_0 = \vec{m}_1 $	6 : Req $T := \vec{\text{pk}}_0^* = \vec{\text{pk}}_1^* = \vec{m} $
7 : $\text{ct} \leftarrow \text{mmPKE.Enc}(\text{pp}, \vec{\text{pk}}_b^*, \vec{m}_b)$	7 : $\text{ct} \leftarrow \text{mmPKE.Enc}(\text{pp}, \vec{\text{pk}}_b^*, \vec{m})$
8 : $b' \leftarrow \mathcal{A}_2^{\text{Dec}^{\text{ATK}}}(\text{st}, \text{ct}, \vec{k})$	8 : $b' \leftarrow \mathcal{A}_2^{\text{Dec}^{\text{ATK}}}(\text{st}, \text{ct}, \vec{k})$
9 : Req $\forall j \in [T] : \vec{\text{pk}}^*[j] \notin (\text{pk}_i)_{i \in [P]}$	9 : Req $\forall j \in [T] : \vec{\text{pk}}_0^*[j] \notin (\text{pk}_i)_{i \in [P]}$
10 : $\implies \vec{m}_0[j] = \vec{m}_1[j]$	10 : $\implies \vec{\text{pk}}_0^*[j] = \vec{\text{pk}}_1^*[j]$
11 : return $b = b'$	11 : return $b = b'$
	Oracle $\text{Dec}^{\text{ATK}}(i, j, c)$ 1 : if $\text{ATK} = \text{CPA} : \text{return } \perp$ 2 : if $c = \text{ct} : \text{return } \perp$ 3 : if $i \in [P] : \text{return } \text{mmPKE.Dec}(\text{pp}, \text{sk}_i, c, j)$ 4 : else return $\perp$

Figure 2: mmPKE security games. $P = \text{poly}(\lambda)$ is the total number of parties in the system, while T is the number of parties per ciphertext.

To encrypt $\ell > 1$ bits $\vec{m}_i \in \{0,1\}^\ell$, recipient i samples $\text{sk}_i \in \mathbb{Z}_q^{n \times \ell}$ and derives $\text{pk}_i \in \mathbb{Z}_q^{n \times \ell}$; we then follow the same encryption procedure as above.

Single-recipient attack. We start with a simple attack, which targets a single public key. Since the public keys could be maliciously formed, the attacker can copy an honest recipient's public key. Now, suppose that the sender wants to send m_i to the honest recipient i and $m_{\mathcal{A}}$ to the adversary, and generates the corresponding ciphertext $(\text{ct}_0, \text{ct}_1, \dots, \text{ct}_T)$, where $\text{ct}_{\mathcal{A}}$ is the ciphertext for the adversary and ct_i is the ciphertext for the honest recipient i . In this case, the adversary can easily know whether $m_{\mathcal{A}} = m_i$, by checking if $\text{ct}_{\mathcal{A}} \approx \text{ct}_i$. This breaks the semantic security.

Although this attack seems easily preventable by requiring all public keys to be unique, it can be generalized to be impossible to prevent as follows. The adversary first generates a short secret $\text{sk}_{\mathcal{A}} \leftarrow \mathcal{D} \in \mathbb{Z}_q^n$, a Gaussian noise $\vec{e}_{\mathcal{A}}$, and computes $\text{pk}_{\mathcal{A}} \leftarrow \text{pk}_i + \text{Ask}_{\mathcal{A}} + \vec{e}_{\mathcal{A}}$. Then, we have $\text{ct}_{\mathcal{A}} \approx \text{ct}_i + \langle \text{ct}_0, \text{sk}_{\mathcal{A}} \rangle$ if and only if $m_{\mathcal{A}} = m_i$, which directly breaks semantic security. Furthermore, based on the LWE assumption, $\text{pk}_{\mathcal{A}}$ is indistinguishable from an honestly generated public key, thus making this attack undetectable.

This attack trivially breaks key privacy as well, since the adversary can easily check whether the attacked public key is one of the public keys used to generate the ciphertext.

Multi-recipient attack. If the adversary is allowed to control multiple recipients, then they can perform more catastrophic attacks.

These more powerful attacks are based on the notion of lattice trapdoors [24]. Intuitively, using

lattice trapdoors one can generate a matrix $\tilde{A} \in \mathbb{Z}_q^{n \times w}$ that is indistinguishable from uniform based on the trapdoor T , such that when given $\tilde{x}\tilde{A} + \tilde{e}$ for short $\tilde{x}$ and $\tilde{e}$, it is easy to recover $\tilde{x}$. For an adversary corrupting w parties, assuming that the encapsulated messages are 0 for those parties, the adversary could use the w columns of $\tilde{A}$ as those corrupted recipients' public keys and thus receive $\tilde{x}\tilde{A} + \tilde{e}$ in the ciphertext. Then using the trapdoor T , the adversary can easily recover $\tilde{x}$, which is the randomness used for the ciphertext, and recover all $T - w$ honest recipients' messages. For gadget trapdoors [42], $w = O(n \log(q))$. For plaintext space $\ell > 1$ bits, the adversary only needs to control $O(n \log(q)/\ell)$ recipients as each recipient can supply ℓ columns of $\tilde{A}$. Since $\tilde{A}$ is indistinguishable from a uniformly randomly sampled matrix, this powerful attack is also *undetectable*.

A similar attack can be performed against key-privacy.

Attacks on mmKEM. Attacking mmKEM instead of mmPKE is more involved, since in the mmKEM setting the attacker cannot control the encrypted messages. I.e., the sender will encrypt a random bit γ_i chosen by itself, instead of encrypting a bit m_i chosen by the adversary.

As in the case of single-recipient attack, the adversary can copy the honest party i 's public key $\text{pk}_{\mathcal{A}} \leftarrow \text{pk}_i$, and check whether $\text{ct}_{\mathcal{A}} \approx \text{ct}_i$ or $\text{ct}_{\mathcal{A}} \approx \text{ct}_i + q/2$, where $(\text{ct}_0, \text{ct}_{\mathcal{A}})$ encrypts $\gamma_{\mathcal{A}}$ for the adversary and $(\text{ct}_0, \text{ct}_i)$ encrypts γ_i for party i . Thus, even though the adversary does not learn $\gamma_{\mathcal{A}}$ explicitly, it knows whether $\gamma_i = \gamma_{\mathcal{A}}$. This suffices for the adversary to win the IND-CPA security game, since it is given either $(\gamma_{\mathcal{A}}, \gamma_i)$ or $(\gamma'_{\mathcal{A}}, \gamma'_i)$, and only needs to decide which pair the ciphertext encrypts. In practice, the adversary may have a way to learn $\gamma_{\mathcal{A}}$ (e.g., by asking the sender via another channel), from which it can deduce the γ_i addressed to party i as above.

Furthermore, the adversary can break key privacy by detecting ciphertext addressed to another user, as follows. The adversary again copies the public key of some honest party i , $\text{pk}_{\mathcal{A}} \leftarrow \text{pk}_i$. It could then tell whether a ciphertext indeed also sends to this party i or not. For example, the adversary obtains a ciphertext targeting to $\tilde{\text{pk}} = (\text{pk}_{\mathcal{A}}, \text{pk}_{i'})$, it checks if $\text{ct}_{\mathcal{A}} \approx \text{ct}_{i'}$ or $\text{ct}_{\mathcal{A}} \approx \text{ct}_{i'} + q/2$. If we have $i' = i$, then the above relation holds with probability close to 1. Otherwise, it happens with probability $O(\frac{\beta}{q})$, where β is the noise bound. For schemes encrypting $\ell > 1$ bits, the latter probability is further decreased to $O((\frac{\beta}{q})^\ell)$. As before, this attack can be generalized to not just copying the public key but instead computing $\text{pk}_{\mathcal{A}} \leftarrow \text{pk}_i + \text{Ask}_{\mathcal{A}} + \vec{e}_{\mathcal{A}}$, rendering it undetectable.

For the multi-recipient attack, it is unclear whether the trapdoor attack directly applies. However, if we do not require $\tilde{A}$ to be indistinguishable from random (thus can potentially be detectable), we can easily invert the randomness by maliciously setting some $\tilde{A}$. For example, if the adversary sets $\tilde{A} = \beta \cdot I$ where I is the identity matrix of size $n \times n$ and β is the Gaussian noise bound (assuming $\beta \ll q$ for simplicity), then it is easy to obtain x , the randomness used for encryption. This type of attack can still be hard to prevent since it can be generalized to other forms of $\tilde{A}$.

(Single-message) Multi-recipient KEM (mKEM). As mentioned in 2, there are mKEM schemes achieving malicious security [28, 26, 2]. These schemes do not suffer from the attacks above because (1) everyone in the group receives the same plaintext (either chosen by the sender in the mPKE case or randomly sampled in the mKEM case); (2) they do not require key privacy.

In [2], the authors construct mmPKE (multi-message multi-recipient) via mKEM. However, in these schemes the recipients who share the same plaintext receive the same encapsulated key and the same DEM ciphertext. In this case, there is some obvious information leakage: the total amount of DEM ciphertexts shows how many different plaintexts there are; and recipients can easily learn who shares the same plaintext as them. Moreover, when everyone has a different plaintext, their construction has no savings compared to naively using regular PKE by generating T independent ciphertexts.

6 Our mmKEM Constructions

In this section, we show two mmKEM constructions: the first construction assumes the security of Oracle Module-LWE (a new Module-LWE variant we introduce below) with a *super-polynomial* modulus-to-noise ratio³, and the second one relies on Oracle Module-LWE with a *polynomial* modulus-to-noise ratio (i.e., regular LWE). These achieve regular correctness (Theorem 4.1) for mmKEM. Later, in Section 8, we show a general compiler to achieve positionless correctness (Theorem 4.2).

6.1 Oracle Module-LWE

Recall that the only existing mmKEM scheme with malicious security is [46]. The assumption they are based on is the so-called *static Diffie-Hellman with oracle* [16, 45]: given a generator g , and fixed group element g^d , and some random g^y , find g^{dy} ; furthermore, the adversary is given an oracle $\mathcal{O}_d(x)$ computing x^d for arbitrary x .

However, there is no counter-part of such an assumption in the lattice world. The reason is obvious as shown in Section 5: to distinguish $(A, A\vec{s} + \vec{e})$ and (A, b) , if the adversary is given an oracle $\mathcal{O}_{\vec{s}}(A')$ outputting $A'\vec{s} + \vec{e}'$ for an arbitrary matrix $A' \in \mathbb{Z}_q^{\ell \times n}$ as hints, the security immediately breaks. The adversary can use the attacks we discussed above to recover $\vec{s}$ thus breaking LWE. However, it seems inherently impossible to build mmKEM with malicious security without allowing the adversary to query some type of oracle.

Nevertheless, it is also obvious that the adversary, given the capability to construct an arbitrary public key, can obtain some hint about the LWE secret. Thus, we need a new lattice assumption that captures this.

Masked hint. To start, the hint has the format $A'\vec{s} + \vec{e}' \in \mathbb{Z}_q^\ell$ where $A' \leftarrow \mathcal{A}$ is a maliciously chosen matrix. Thus, the matrix can be arbitrary. The first idea is to mask the hint with $\vec{u}$ where $\vec{u} \xleftarrow{\$} \mathbb{Z}_q^\ell$ (i.e., output $A'\vec{s} + \vec{e}' + \vec{u}$). In this case, the uniformly sampled $\vec{u}$ serves as a one-time pad to mask $A'\vec{s} + \vec{e}'$. With this, the hints do not provide any information in a trivial way.

Adding a random oracle. However, obviously, in this case the hint is uniformly random which contains no information, while the mmKEM scheme needs additional information to guarantee that honest users can correctly decapsulate their keys. To allow for decapsulation, we provide additional information: together with $A'\vec{s} + \vec{e}' + \vec{u}$ the hint contains $H(\lfloor p \cdot \vec{u} / q \rfloor)$ where $H : \mathbb{Z}_p^\ell \rightarrow \mathcal{K}$ is a random oracle for the key space $\mathcal{K}$ (for simplicity, imagine $\mathcal{K}$ being $\{0,1\}^\lambda$ for some security parameter λ) and some $p \leq q$. We add the p parameter for additional flexibility which later in our construction serves as the plaintext space.

The challenge samples. Given such hints, it is in fact still very easy to distinguish an LWE sample $A \in \mathbb{Z}_q^{n \times n}$, $\vec{b} \leftarrow A\vec{s} + \vec{e} \in \mathbb{Z}_q^n$ from $(A, \vec{b})$ for uniformly randomly sampled $\vec{b}$, for $p \ll q$.

This challenge can be differentiated using the following attack. The attack first samples $\vec{x} \xleftarrow{\$} (1, 0, 0, \dots, 0)$ and computes $\vec{a} \leftarrow \vec{x}A$, $b \leftarrow \langle \vec{x}, \vec{b} \rangle$. Then, with hint $a \leftarrow \langle \vec{a}, \vec{s} \rangle + e + u \in \mathbb{Z}_q$ (WLOG, assume $\ell = 1$, but this works for any $\ell > 0$), the adversary computes $c \leftarrow a - b$. If the challenge is a valid LWE sample, we have that $c = u + e - \vec{e}'[1]$ and as long as $q/p > |e - \vec{e}'[1]|$, $\lfloor p(a - b)/q \rfloor = \lfloor pu/q \rfloor$, and whether this is true can be tested using the random oracle. Otherwise, $\lfloor p(a - b)/q \rfloor \neq \lfloor pu/q \rfloor$ except with a small probability.

³The first construction serves as a stepping stone, and also has slightly faster decapsulation time compared to the second construction.

Our final assumption. Instead of asking the adversary to distinguish regular LWE samples from uniform, we ask the adversary to distinguish “masked” LWE samples from uniform. In more detail, we ask the adversary given an LWE sample $(A_1, A_1 \vec{s} + \vec{e})$ together with the masked hints $A' \vec{s} + \vec{e}' + \vec{u}', H(\lfloor p \cdot \vec{u}' / q \rfloor)$ for *any maliciously chosen* $A' \in \mathbb{Z}_q^{w \times n}$ to distinguish between (1) $(A_2, A_2 \vec{s} + \vec{e}_2 + \vec{u}, H(\lfloor p \cdot \vec{u} / q \rfloor))$, and (2) $(A_2, \vec{b}, H(\lfloor p \cdot \vec{u} / q \rfloor))$; where $\vec{s} \xleftarrow{\$} \mathbb{Z}_q^n$, $A_1 \xleftarrow{\$} \mathbb{Z}_q^{w_1 \times n}$, $\vec{e} \leftarrow \chi^{w_1}$, $\vec{e}' \leftarrow \chi^w$, $\vec{u}' \xleftarrow{\$} \mathbb{Z}_q^w$, $A_2 \xleftarrow{\$} \mathbb{Z}_q^{w_2 \times n}$, $\vec{e}_2 \leftarrow \chi^{w_2}$, $\vec{u} \xleftarrow{\$} \mathbb{Z}_q^{w_2}$, and $\vec{b} \xleftarrow{\$} \mathbb{Z}_q^{w_2}$ for any $w_1, w_2 = \text{poly}(\lambda)$.

Interestingly, this also makes our assumption similar to the so-called one-more Diffie-Hellman assumption [8] and one-more SIS assumption [1], which require the adversary to distinguish/generate one additional instance after querying a polynomial amount of instances. Similarly, our assumption allows the adversary to ask an arbitrarily polynomial amount of masked LWE samples, in order to distinguish an honestly generated masked LWE sample from a random vector.

With this intuition in mind, we now formalize our new Module-LWE variant (naturally extending the idea for LWE above), Oracle Module-LWE in Theorem 6.1. In Section 7, we provide some cryptanalysis attempts, showing (1) some parameters for which the assumption breaks in Section 7.1, (2) some parameters for which the assumption holds under standard MLWE in Section 7.2, and (3) intuition on why the assumption holds for parameters of interest in Section 7.3.

Definition 6.1 (Decisional Oracle Module-LWE). Let $n, N, q, \mathcal{D}, \chi, p$ be parameters dependent on λ and N being a power of two. Let $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $H : \mathcal{R}_p \rightarrow \mathcal{K}$ be a random oracle. The oracle module learning with error (OMLWE) problem $\text{OMLWE}_{n, N, q, \mathcal{D}, \chi, p, H}$ is the following: for any $w_1, w_2, w_{\mathcal{A}} = \text{poly}(\lambda)$, let $A_1 \xleftarrow{\$} \mathcal{R}_q^{w_1 \times n}$, $A_2 \xleftarrow{\$} \mathcal{R}_q^{w_2 \times n}$ over $\mathcal{R}_q$, distinguish the following two distributions (difference marked in blue)

$$\begin{aligned} & (A_2, \textcolor{blue}{A_2 \vec{s} + \vec{e}_2 + \vec{u}_2}, (H(\lfloor p \cdot \vec{u}_2[i] / q \rfloor)))_{i \in [w_2]}, \\ & (A_1, A_1 \vec{s} + \vec{e}_1), \\ & (A_{\mathcal{A}}, A_{\mathcal{A}} \vec{s} + \vec{e}_{\mathcal{A}} + \vec{u}_{\mathcal{A}}, (H(\lfloor p \cdot \vec{u}_{\mathcal{A}}[i] / q \rfloor)))_{i \in [w_{\mathcal{A}}]} \end{aligned}$$

from

$$\begin{aligned} & (A_2, \vec{r}_2, (H(\lfloor p \cdot \vec{u}_2[i] / q \rfloor)))_{i \in [w_2]}, \\ & (A_1, A_1 \vec{s} + \vec{e}_1), \\ & (A_{\mathcal{A}}, A_{\mathcal{A}} \vec{s} + \vec{e}_{\mathcal{A}} + \vec{u}_{\mathcal{A}}, (H(\lfloor p \cdot \vec{u}_{\mathcal{A}}[i] / q \rfloor)))_{i \in [w_{\mathcal{A}}]} \end{aligned}$$

with noticeable advantage, where $\vec{s} \xleftarrow{\$} \mathcal{D}^n$, $A_{\mathcal{A}} \leftarrow \mathcal{D}_{\mathcal{A}}$, $\vec{u}_2 \xleftarrow{\$} \mathcal{R}_q^{w_2}$, $\vec{u}_{\mathcal{A}} \xleftarrow{\$} \mathcal{R}_q^{w_{\mathcal{A}}}$, $\vec{e}_1 \leftarrow \chi^{w_1}$, $\vec{e}_2 \leftarrow \chi^{w_2}$, $\vec{e}_{\mathcal{A}} \leftarrow \chi^{w_{\mathcal{A}}}$, and $\vec{r}_2 \xleftarrow{\$} \mathcal{R}^{w_1}$.

6.2 First Construction with Superpolynomial Modulus-to-noise Ratio

Our first goal is to construct an encryption scheme that does not suffer from the attacks described in Section 5. As hinted above, we use OMLWE to achieve this. The underlying issue in these attacks is that for a ciphertext $\text{ct} := (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_T)$, $\text{ct}_1, \dots, \text{ct}_T$ are potentially dependent on maliciously-generated public keys, thus being identical to obtaining the masked samples in OMLWE. We now show how exactly we leverage OMLWE to build mmKEM.

Main technique. Like many lattice-based constructions, we encrypt the message in the most significant bits of the ciphertext, and set $q = p \cdot r$, where p is the plaintext space and r is the buffer

to accommodate noise. Specifically following the notation of Section 5, to construct a ciphertext $\text{ct} := (\text{ct}_0, \text{ct}_1) \in \mathbb{Z}_q^n \times \mathbb{Z}_q^\ell$ (i.e., encrypting $\ell = O(\lambda)$ bits)⁴ with respect to public key pk , we compute $\text{ct}_0 = \vec{x}A + \vec{e}_1$ and $\text{ct}_1 = \vec{x}\text{pk} + \vec{u} \cdot r + \vec{y} + \vec{e}'$ where $\vec{e}_1, \vec{e}'$ are Gaussian noise, $\vec{u} \xleftarrow{\$} \mathbb{Z}_p^\ell$ is a uniform vector, and $\vec{y} \xleftarrow{\$} \mathbb{Z}_r^\ell$ is a uniform noise. Notice that $\vec{u} \cdot r + \vec{y}$ is uniformly random in $\mathbb{Z}_q^\ell$, which is exactly the same distribution as the hints in OMLWE (Theorem 6.1).

Now, it remains to construct a key encapsulation procedure using the above encryption scheme. Recall that in the security game of KEM (Fig. 1), the adversary tries to distinguish the encapsulated key vector $\vec{k}_0$ from a random vector $\vec{k}_1$. As seen in Section 5, if the adversary establishes some correlations between the encapsulated keys, it can distinguish the two cases. To address this, we set the final key to be $H(\vec{u})$, where H is a random oracle. In this case, even knowing $H(\vec{u})$, $\vec{u}$ looks still uniform to the adversary (recall that $\ell = \lambda$, so there are no collisions among different recipients in the case of multiple recipients except for negligible probability).

An additional problem arises due to the noise accumulation in the ciphertexts. Specifically, given a ciphertext $(\text{ct}_0, \text{ct}_1)$ the decryption procedure computes $\text{ct}_0 \cdot \text{sk} - \text{ct}_1 = \vec{u} \cdot r + \vec{E}$, where $\vec{E} = \vec{E}_g + \vec{y}$ such that $\vec{E}_g$ is the total Gaussian noise (induced by the Gaussian noise in the public key and in ct_0, ct_1) and $\vec{y}$ is uniform noise used in the encapsulation procedure. Observe that for $\vec{y} \xleftarrow{\$} \mathbb{Z}_r^\ell$, the noise $\vec{E}_g + \vec{y}$ might cause an overflow and break the correctness guarantee. To mitigate this issue, we bound the size of the noise $\vec{E}_g$ by $B = O(\lambda \cdot \sigma \cdot \beta)$, where λ is the security parameter, σ is the individual Gaussian noise bound, and β is the norm bound of the secret key. Then, as long as we choose r such that $B/r = \text{negl}(\lambda)$, an overflow happens with negligible probability, allowing $\vec{y}$ to be uniformly sampled.

This naturally extends to multiple parties (each with their own public key and encapsulated key), and the module setting, which yields our first construction, formalized in Algorithm 2.

Theorem 6.2. *For any $\lambda > 0, p \geq 2$, and $\mathcal{K}$, assuming that $\text{MLWE}_{n,N,q,\mathcal{D}_\beta,\sigma}$ and $\text{OMLWE}_{n,N,q,\mathcal{D}_\beta,\sigma,p,H}$ holds, mmKEM1 (Algorithm 2) is a IND-CPA-secure and KP-CPA-secure multi-message multi-recipient KEM (Fig. 1) with regular correctness (Theorem 4.1).*

Proof. We show the properties separately.

- (Regular correctness, Theorem 4.1) We start by calculating the decapsulated key obtained in the Decaps procedure:

$$\begin{aligned} \vec{u}'_i &\leftarrow \lfloor (b_i - \vec{a}'_i \text{sk}_i) / r \rfloor = \lfloor (\vec{x}^T \text{Ask}_i + \vec{e}) + \vec{y}_i + \vec{u}_i \cdot r - (\vec{x}^T A + \vec{e}_1) \text{sk} \rfloor / r \\ &= \lfloor (\vec{u}_i \cdot r + \vec{x}^T \vec{e} + \vec{e}'_i + \vec{y}_i - \vec{e}_1 \text{sk}) / r \rfloor \\ &= \vec{u}_i + \lfloor (\vec{x}^T \vec{e} + \vec{e}'_i + \vec{y}_i - \vec{e}_1 \text{sk}) / r \rfloor \in \mathbb{Z}_p^\ell \end{aligned}$$

To argue correctness, it suffices to show that for each recipient i , with all but negligible probability, $\vec{u}'_i = \vec{u}_i$ (and thereby $H(\vec{u}'_i) = H(\vec{u}_i) = k_i$). Note that this is independent for each i , as seen from Algorithm 2. It holds that $\vec{u}'_i = \vec{u}_i$ if and only if for all $j \in [N]$, $(\vec{x}^T \vec{e} + \vec{e}'_i + \vec{y}_i - \vec{e}_1 \text{sk})[j] \in [0, r)$. Note that $\vec{y}_i[j]$ is sampled uniformly at random from $[0, r)$. Therefore, a naive bound of the norm of $(\vec{x}^T \vec{e} + \vec{e}'_i + \vec{y}_i - \vec{e}_1 \text{sk})[j]$ is $[-B, r + B)$, where $B := N \cdot n \cdot |x| \cdot |e| + N \cdot n \cdot |\text{sk}| \cdot |e_1| + |e'| = O(N \cdot n |\chi_\sigma| |\mathcal{D}_\beta|)$. However, note that $\frac{B}{r} = \text{negl}(\lambda)$ (as required by line 3 in Algorithm 2). Thus, the statistical distance between the

⁴For smaller key space, we can simply enforce using a key space of at least λ bits.

Algorithm 2 mmKEM1

```

1: procedure mmKEM1.GenParam( $1^\lambda, \text{aux} = (p, \mathcal{K})$ )
2:   Let  $\ell \leftarrow \lceil \max(\log(|\mathcal{K}|), \lambda) / \log(p) \rceil$  i.e.,  $\ell$  is the minimum integer such that  $p^\ell \geq |\mathcal{K}|$  or  $p^\ell$ 
   has  $\lambda$  bits
3:   Choose the ring dimension  $N \geq \ell$ , secret key length  $n$ , and secret key distribution  $\mathcal{D}_\beta$ ,
   Gaussian noise with standard deviation  $\sigma$ , and uniform noise bound  $r$  such that  $\frac{N \cdot n \cdot |\chi_\sigma| \cdot |\mathcal{D}_\beta|}{r} = \text{negl}(\lambda)$ 
4:   Set ciphertext modulus  $q := p \cdot r$ 
5:   Sample the shared common ring matrix  $A \xleftarrow{\$} \mathcal{R}_q^{n \times n}$ 
6:   Initialize a function  $H : \mathbb{Z}_p^N \rightarrow \mathcal{K}$  (or  $\mathbb{Z}_p^N \rightarrow \{0,1\}^\lambda$  if  $|\mathcal{K}| < 2^\lambda$ ).
7:   return  $\text{pp} = (\lambda, p, \mathcal{K}, \ell, N, \mathcal{D}_\beta, \sigma, r, q, A, H)$ 

8: procedure mmKEM1.KeyGen( $\text{pp}$ )
9:    $\text{sk} \xleftarrow{\$} \mathcal{D}_\beta^n \in \mathcal{R}^n, \vec{e} \leftarrow \chi_\sigma^n \in \mathcal{R}^{n \times 1}$ 
10:   $\text{pk} := \text{Ask} + \vec{e} \in \mathcal{R}_q^{n \times 1}$ 
11:  return  $(\text{pk}, \text{sk})$ 

12: procedure mmKEM1.Encapsind( $\text{pp}; r_0$ )
13:   $\vec{x} \leftarrow_{r_0} \mathcal{D}_\beta^{1 \times n} \in \mathcal{R}^{1 \times n}, \vec{e}_1 \leftarrow_{r_0} \chi_\sigma^{1 \times n}$ 
14:   $\vec{a}' \leftarrow \vec{x}A + \vec{e}_1$ 
15:  return  $\text{ct}_0 \leftarrow \vec{a}' \in \mathcal{R}_q^{1 \times n}$ 

16: procedure mmKEM1.Encapsdep( $\text{pp}, \text{pk}_i; r_0, r_i$ )
17:   $\vec{u}_i \xleftarrow{\$}_{r_i} \mathbb{Z}_p^N$ 
18:   $\vec{x} \leftarrow_{r_0} \mathcal{D}_\beta^n$ 
19:   $\vec{y}_i \xleftarrow{\$}_{r_i} \mathbb{Z}_r^N$ 
20:   $e'_i \leftarrow \chi_\sigma$ 
21:   $b_i \leftarrow \vec{x}\text{pk}_i + \vec{u}_i \cdot r + \vec{y}_i + e'_i \in \mathcal{R}$   $\triangleright$  The addition between  $\mathcal{R}_q$  and  $\mathbb{Z}_p^N$  elements is done as
   defined in Section 3.1.
22:  return  $\text{ct}_i \leftarrow b_i, k_i \leftarrow H(\vec{u})$ 
    $\triangleright$  Recall that mmKEM1.Encaps calling Encapsind and Encapsdep is defined in Algorithm 1

23: procedure mmKEM1.Decaps( $\text{pp}, \text{sk}, \text{ct} = (\text{ct}_0 = a', \text{ct}_1 = b_1, \dots, \text{ct}_T = b_T), i$ )
24:   $\vec{u}' \leftarrow \lfloor (b_i - a'\text{sk})/r \rfloor \in \mathbb{Z}_p^N$ 
25:  return  $H(\vec{u}')$ 

```

distribution of $(\vec{x}^T \vec{e} + e'_i + \vec{y}_i - \vec{e}_1^T \text{sk})[j]$ and the uniform distribution over $[0, r)$ is negligible, by the smudging lemma [3, Lemma 2.1]. Therefore, $\vec{u}' = \vec{u}$, except with $\text{negl}(\lambda)$ probability.

- (IND-CPA security) We argue the scheme's IND-CPA security via a hybrid argument over the following games and show that all games are indistinguishable, and eventually for the very last hybrid game, the adversary cannot have more than $1/2$ advantage.
 - hyb_0 : same as the CPA game in Figure 1 using our construction in Algorithm 2.
 - hyb_1 : same as hyb_0 except that all honest public keys are sampled as $\text{pk}_j \xleftarrow{\$} \mathcal{R}_q$ instead of honestly generated.

- hyb_2 : same as hyb_1 except that $b_j \xleftarrow{\$} \mathbb{Z}_q^N$ for non-corrupted keys pk_j , and $a' \xleftarrow{\$} \mathcal{R}_q$ (the ciphertext components output of Encaps).

It is clear that hyb_2 is IND-CPA secure, since (1) the ciphertext is simply independent of the keys, and (2) pk is independent of sk .

We first argue that hyb_2 and hyb_1 are indistinguishable. This is based on $\text{OMLWE}_{n,N,q,\mathcal{D}_\beta,\sigma,p,H}$: in hyb_1 , all honest public keys are replaced by uniform ring elements, and thus the adversary $\mathcal{A}$ distinguishing the two hybrids can be used to build $\mathcal{A}'$ to break $\text{OMLWE}_{n,N,q,\mathcal{D}_\beta,\sigma,p,H}$.

In more detail, let $w_1 = n$ and $w_2 = P$ (the total number of public keys), $\mathcal{A}'$ first obtains A_1 and A_2 , and uses A_1 as the common matrix A in pp of the encapsulation scheme and A_2 as public keys for all honest users; then, $\mathcal{A}$ sends back the maliciously generated public keys, denoted as $A_{\mathcal{A}}$. Finally, $\mathcal{A}'$ receives the OMLWE sample:

$$\begin{aligned} & (A_2, \vec{b}_2, (H(\lfloor p \cdot \vec{u}_2[i]/q \rfloor))_{i \in [w_2]}), \\ & (A_1, A_1 \vec{s} + \vec{e}_1), \\ & (A_{\mathcal{A}}, A_{\mathcal{A}} \vec{s} + \vec{e}_{\mathcal{A}} + \vec{u}_{\mathcal{A}}, (H(\lfloor p \cdot \vec{u}_{\mathcal{A}}[i]/q \rfloor))_{i \in [w_{\mathcal{A}}]}) \end{aligned}$$

Upon receiving the challenge request from $\mathcal{A}$, the adversary $\mathcal{A}'$ returns $A_1 \vec{s} + \vec{e}_1$ as ct_0 , and ct_i is either the i -th element $\vec{b}_2$ or $A_{\mathcal{A}} \vec{s} + \vec{e}_{\mathcal{A}} + \vec{u}_{\mathcal{A}}$, depending on whether the public key is honestly or maliciously generated respectively.

If $\vec{b}$ is uniform, then this is identical to hyb_2 . If $\vec{b}$ is $A_2 \vec{s} + \vec{e}_2 + \vec{u}_2$, then this is identical to hyb_1 . Thus, if $\mathcal{A}$ distinguishes hyb_1 with hyb_2 , $\mathcal{A}'$ breaks the corresponding OMLWE assumption.

Then hyb_1 and hyb_0 are indistinguishable based on $\text{MLWE}_{N,q,\mathcal{D}_\beta,\sigma}$, as $\text{pk} := \text{ask} + e$ in hyb_0 is a valid MLWE sample. Thus, we conclude that our scheme is IND-CPA secure.

- (CPA-key-privacy) For CPA-key-privacy, we simply see that hyb_2 is CPA-key-private for the same reason as above. Therefore, the same argument shows that our scheme is also CPA-key-private.

□

6.3 Second Construction with Polynomial Modulus-to-noise Ratio

While the above construction already satisfies our initial goal of building a lattice-based mmKEM scheme with malicious security, there is one major disadvantage: its security argument is based on Oracle MLWE with a superpolynomial modulus-to-noise ratio, i.e., $\text{OMLWE}_{n,N,q,\mathcal{D}_\beta,\sigma,p,H}$ with $\frac{|\chi\sigma|}{q} = \text{negl}(\lambda)$. Even though there is no known attack on these parameters of Oracle-MLWE, it is theoretically a much stronger assumption than Oracle-MLWE with a polynomial modulus-to-noise ratio. Furthermore, it results in a relatively bad concrete efficiency, since q needs to be superpolynomial in λ . This means that q needs to be large (e.g., $> 2^{40}$), and hence for any fixed σ , the ring dimension N also needs to be large for the MLWE assumption to hold, which in practice significantly increases the resulting ciphertext size.

Weaker correctness guarantee. We observe that in the construction of Section 6.2, q needs to be large only for correctness, and that the security analysis remains valid even when q is polynomial.

In more detail, recall that the decapsulation error of the intermediate key vector (i.e., $\vec{u}$) is $E := b_i - (a' \text{sk}) - r\vec{u} = x \cdot e + \vec{y} - \text{sk} \cdot e_1 \in \mathbb{Z}_q^\ell$. If $q = \text{poly}(\lambda)$, then it must be that $r = \text{poly}(\lambda)$, and hence the probability that $E[j]$ is larger than r is at least $\Pr[(x \cdot e - \text{sk} \cdot e_1)[j] > 0] \cdot \Pr[\vec{y}[j] = r - 1] = 1/\text{poly}(\lambda)$. So, $E[j] \notin [0, r)$ with probability at least $1/\text{poly}(\lambda)$, which is non-negligible. Note that $E[j] \in [-B, r + B)$, where B is the noise bound of the Gaussian noise, so the key value can only be shifted by one (i.e., $\vec{u}'[j] \in [\vec{u}[j] - 1, \vec{u}[j] + 1]$).

To ease the analysis of noise overflow, we shift the noise distribution from χ_σ to $\chi_{+, \sigma}$, and the secret distribution from $\mathcal{D}_\beta$ to $\mathcal{D}_{+, \beta}$ (see Section 3.1). In this way, all Gaussian noises are positive and thus $E[j] \geq 0$ for all $j \in [\ell]$ (except with negligible probability)⁵. This means that with probability $P = 1/\text{poly}(\lambda)$, the key value would be shifted up by one to $\vec{u}'[j] \leftarrow \vec{u}[j] + 1$.

Correcting the errors. A natural idea is to use error-correcting codes to correct the decapsulation errors. Unfortunately, if we encode $\vec{u}$ using error-correcting codes, the codeword is no longer pseudorandom, which breaks our security argument (i.e., hyb_3 is no longer indistinguishable from hyb_2 in the proof of Theorem 6.2)⁶.

Instead, we observe that by choosing r, σ, β, N suitably, we can let $P = \Omega(1/\lambda)$ and thus $P^{\log \log(\lambda)} = \text{negl}(\lambda)$. Therefore, such a “shifted-up-by-one” error can happen to *at most* $(\log \log(\lambda))$ -out-of- ℓ elements⁷. Let us first assume that the recipient can test whether a key is indeed correct (we will explain how to do that next). Then intuitively, the recipient could test all possible “shifted-up-by-one” error until the correct key is found. However, this naive approach requires $\binom{\ell}{\log \log(\lambda)}$ trials, which is super-polynomial in λ , since $\ell = \lceil \lambda / \log(p) \rceil$ (as key space size $|\mathcal{K}| \geq 2^\lambda$) and $p \leq q = \text{poly}(\lambda)$.

Luckily, the recipient can in fact tell which elements are potentially wrong by looking at the noise before rounding: if an error occurs (i.e., the decoded key is shifted up by one), we would have $E[j] \bmod r = ((b_i - a' \text{sk} - r\vec{u})[j] \bmod r) \in [0, B)$, where $B = O(N|\chi_\sigma||\mathcal{D}_\beta|)$ is the noise bound of the accumulative noise except for $\vec{y}[j] \stackrel{\$}{\leftarrow} [0, r)$. Thus, the recipient only needs to test the coordinates where $E[j] \bmod r \in [0, B)$. The recipient identifies all such elements, which are at most $O(\log \log(\lambda))$ except for $\text{negl}(\lambda)$ probability, and tests for correctness of keys containing either $\vec{u}[j] + 1$ or $\vec{u}[j]$ (because $E[j]$ might belong in $[0, B)$, even though $\vec{u}[j]$ is the correct key element, which is indistinguishable from $E[j] \in [r, B)$ and the key is shifted up by one). If each key correctness test takes T time, the above procedure terminates in $T \cdot 2^{O(\log \log(\lambda))} = T \cdot \text{polylog}(\lambda)$ time.

Testing the key. We now explain how to enhance the encapsulation procedure to allow for an efficient testing strategy for testing the correctness of the key. During encapsulation, the sender produces $c \leftarrow G(\vec{u})$, where G is another random oracle different from H , and attaches c to the ciphertext. The recipient then evaluates $c' \leftarrow G(\vec{u}')$ on all $2^{O(\log \log(\lambda))}$ possible key vectors $\vec{u}'$ and checks if $c' = c$. Evaluating a hash (modeled as a random oracle) is very efficient, taking $O(\lambda)$ time,

⁵Such shift does not affect security since given a MLWE sample $(a, b = a\text{sk} + e)$ where $\text{sk} \leftarrow \mathcal{D}_\beta$ and $e \leftarrow \chi_\sigma$, we can easily generate $b' \leftarrow b + \beta \cdot a + |\chi_\sigma|$. The tuple (a, b') is a new MLWE sample with the secret and error shifted to positive values except with negligible probability.

⁶One may wonder whether a pseudo-random error-correcting code works [18]. While it indeed works, it is unfortunately quite heavy and thus greatly degrades the efficiency of our construction.

⁷Having $> \log \log(\lambda)$ elements is bounded by $(\ell - \log \log(\lambda)) \cdot P^{\log \log(\lambda)} = \text{negl}(\lambda)$ by union bound.

so the total time is simply $O(\lambda \text{polylog}(\lambda))$.⁸

The final construction is formalized in Algorithm 3 (differences from Algorithm 2 are highlighted in blue).

Theorem 6.3. *For any $\lambda > 0, p \geq 2, \mathcal{K}$, given that $\text{MLWE}_{n,N,q,\mathcal{D}_{\beta},\sigma}$ and $\text{OMLWE}_{n,N,q,\mathcal{D}_{\beta},\sigma,p,H}$ hold and the functions H and G are modeled as random oracles, mmKEM2 (Algorithm 2) is a maliciously IND-CPA-secure and KP-CPA-secure multi-message multi-recipient KEM (Fig. 1) with regular correctness (Theorem 4.1).*

Proof. Note that the IND-CPA and CPA-key-privacy properties hold in the exact same way as Theorem 6.2, so we will only argue about the correctness.

Recall that for recipient i , with the expected corresponding ciphertext $(\text{ct}_0 = \vec{a}', \text{ct}_i = (b_i, c_i))$, the expected decapsulated key is $k_i \leftarrow H(\vec{u})$. Let $E \leftarrow (b_i - \vec{a}'\text{sk}) - \vec{u} \cdot r \in [0, r + B)$ denote the total noise with respect to the correct (intermediate) key $\vec{u}$.

We first argue that if $E[j] \in [B + 1, r)$, then $\vec{u}'[j] = \vec{u}[j]$. In other words, if the noise $E[j]$ is large enough, the extracted key element $\vec{u}'[j]$ is errorless. This is because $\vec{u}'[j]$ is computed by rounding down, as in line 27, so any noise $< r$ is not overflowing and gets removed by rounding down correctly.

Otherwise, if $E[j] \notin [B + 1, r)$, we have either $\vec{u}'[j] = \vec{u}[j]$ when $E[j] \in [0, B]$ or $\vec{u}'[j] = \vec{u}[j] + 1$ when if $E[j] \in [r, B + r)$ before performing the rounding down. Hence, S (defined in line 28) includes all possible erred positions with probability $1 - \text{negl}(\lambda)$, which means that there exists $s \subseteq S$ such that $\vec{u} = \vec{u}'_s$. The recipient can correctly identify this particular s unless there exists some s' such that $\vec{u}'_{s'} \neq \vec{u}'_s$ and $G(\vec{u}'_{s'}) = G(\vec{u})$, which happens with $1/2^\lambda$ probability, assuming G is a random oracle. Thus, correctness holds.

Lastly, we argue that the efficiency of **Decaps** (i.e., can be computed in $\text{poly}(\lambda)$ time). To argue this, let X denote the number of all possible subsets of S (i.e., $X := 2^{|S|}$), and we argue that $X = \text{polylog}(\lambda)$. Since $\frac{B}{r} \leq 1/\lambda$, we know that $P = \Pr[E[j] \leq B] = \Pr[\vec{y}[j] \in [B + 1, r)] = O(1/\lambda)$ for all $j \in [\ell]$. Thus, $|S| = O(\log \log(\lambda))$, except for $P^{|S|} = \text{negl}(\lambda)$ probability. Thus, $X = 2^{|S|} = \text{polylog}(\lambda)$. Since computing G can be computed efficiently (in $O(\lambda)$ time), **Decaps** runs in $O(\lambda \text{polylog}(\lambda))$ time. $\square$

Remark 6.4. The scheme could be extended to Ring-LWE *without* paying the price of having the full ring element: in other words, if one directly uses our scheme with RLWE, i.e., by setting $n = 1$, the ciphertext size is relatively large since b_i is a full ring element. However, instead, we can simply output $b_i[1 : \ell]$. This, of course, requires the domain of H, G to be changed to $\mathbb{Z}_p^\ell$ from $\mathbb{Z}_p^N$ for ring dimension N . Furthermore, the assumption OMLWE also needs to be adapted to reflect this. Essentially, the assumption is not only adapted to fixed $n = 1$, but also require that instead of outputting the entire masked sample $(b \leftarrow as + e + u, H(\lfloor pu/q \rfloor))$, it only outputs $(b[1 : \ell], H(\lfloor p \cdot u[1 : \ell]/q \rfloor))$ (i.e., use the first ℓ coefficients of b and u). This makes the assumption more complicated, and thus we do not explicitly describe this optimization. However, we believe that this does not hurt the security guarantees.

⁸Alternatively, the sender can use a DEM and a correct key to encrypt a value zero. The recipient uses the DEM to test if the key is correct. Since this is slightly more involved and requires that the underlying DEM can test whether a key is correct, we leave it as an alternative.

Algorithm 3 mmKEM2

```

1: procedure mmKEM2.GenParam( $1^\lambda, \text{aux} = (p, \mathcal{K})$ )
2:   Let  $\ell \leftarrow \lceil \max(\log(|\mathcal{K}|), \lambda) / \log(p) \rceil$  ▷ i.e.,  $\ell$  is the minimum integer such that  $p^\ell \geq |\mathcal{K}|$  or  $p^\ell$ 
   has  $\lambda$  bits
3:   Choose the ring dimension  $N \geq \ell$ , secret key length  $n$ , and secret key distribution  $\mathcal{D}_{+, \beta}$ ,
   Gaussian noise standard deviation  $\sigma$ , and uniform noise bound  $r$  such that  $\frac{N(2|\chi_\sigma|+1)(2|\mathcal{D}_\beta|+1)}{r} \leq 1/\lambda$ 
4:   Set ciphertext modulus  $q = p \cdot r$ 
5:   Sample the shared common ring matrix  $A \xleftarrow{\$} \mathcal{R}_q^{n \times n}$ 
6:   Initialize a function  $H : \mathbb{Z}_p^N \rightarrow \mathcal{K}$  (or  $\mathbb{Z}_p^N \rightarrow \{0,1\}^\lambda$  if  $|\mathcal{K}| < 2^\lambda$ ).
7:   Initialize a function  $G : \mathbb{Z}_p^N \rightarrow \{0,1\}^\lambda$  (or  $\{0,1\}^\lambda \rightarrow \{0,1\}^\lambda$  if  $|\mathcal{K}| < 2^\lambda$ ).
8:    $V := \{a' \in \mathcal{R}_q \mid v \text{ contains at most half zero coefficients}\}$ 
9:   return  $\text{pp} = (\lambda, p, \mathcal{K}, \ell, N, \mathcal{D}_\beta, \sigma, r, q, A, H, G)$ 

10: procedure mmKEM2.KeyGen(pp)
11:    $\text{sk} \xleftarrow{\$} \mathcal{D}_{+, \beta} \in \mathcal{R}_q^n, e \leftarrow \chi_{+, \sigma}^n \in \mathcal{R}^{n \times 1}$ 
12:    $\text{pk} := A\text{sk} + \vec{e} \in \mathcal{R}_q^{n \times 1}$ 
13:   return  $(\text{pk}, \text{sk})$ 

14: procedure mmKEM1.Encapsind(pp;  $r_0$ )
15:    $\vec{x} \leftarrow_{r_0} \mathcal{D}_{+, \beta}^n \in \mathcal{R}_q^n, \vec{e}_1 \leftarrow_{r_0} \chi_{+, \sigma}^n$ 
16:    $\vec{a}' \leftarrow \vec{x}A + \vec{e}_1$ 
17:   return  $\text{ct}_0 \leftarrow \vec{a}' \in \mathcal{R}_q^{1 \times n}$ 

18: procedure mmKEM1.Encapsdep(pp,  $\text{pk}_i; r_0, r_i$ )
19:    $\vec{u}_i \xleftarrow{\$}_{r_i} \mathbb{Z}_p^N$ 
20:    $\vec{x} \leftarrow_{r_0} \mathcal{D}_{+, \beta}^n$ 
21:    $\vec{y}_i \xleftarrow{\$}_{r_i} [0, r-1]^\ell$ 
22:    $e'_i \leftarrow \chi_\sigma$ 
23:    $b_i \leftarrow \vec{x}\text{pk}_i + \vec{u}_i \cdot r + \vec{y}_i + e'_i \in \mathcal{R}$ 
24:    $c_i \leftarrow G(\vec{u}_i)$ 
25:   return  $\text{ct}_i \leftarrow (b_i, c_i) \in \mathbb{Z}_q^\ell \times \{0,1\}^\lambda, k_i \leftarrow H(\vec{u}_i)$ 

26: procedure mmKEM2.Decaps(pp, sk, ct,  $i$ )
27:    $\vec{u}' \leftarrow \lfloor (b_i - a'\text{sk})/r \rfloor$ 
28:   Define set  $S := \{j \in [\ell] : (b_i - a'\text{sk} - \vec{u}' \cdot r)[j] \leq B\}$  where  $B = N(2|\chi_\sigma| + 1)(2|\mathcal{D}_\beta| + 1)$ .
29:   for  $s \subseteq S$  do ▷ Including the empty set
30:      $\vec{u}'_s \leftarrow \vec{u}'$ 
31:     for  $j \in s$  do
32:        $\vec{u}'_s[j] \leftarrow \vec{u}'_s[j] - 1 \in \mathbb{Z}_p$ 
33:     If  $G(\vec{u}'_s) = c_i$ ,  $\vec{u}' \leftarrow \vec{u}'_s$  and break the loop.
34:   return  $H(\vec{u}')$ 

```

6.4 Additional Discussion

mmPKE. Similar to a regular KEM to PKE compiler, our mmKEM to mmPKE compilation is straightforward. The sender uses the mmKEM to agree on keys with each recipient respectively, and then it uses the keys with a data encapsulation mechanism (DEM, Section 3.3) to encrypt the messages. The recipients use mmKEM to decapsulate the keys and use the corresponding keys to decrypt the DEM ciphertexts. Notice that, unlike prior works [28, 26, 2] where all recipients obtain the same key via mKEM, encapsulated keys can be different in our construction, and thus the security of our mmPKE is straightforwardly implied by the security of mmKEM. This transformation is straightforward (essentially identical to the one in [46]). For completeness, we formally present it in Algorithm 4.

Algorithm 4 mmKEM to mmPKE Compiler

```

1: procedure CompilerKEMtoPKE.GenParam( $1^\lambda, \text{aux}$ )
2:    $\text{pp}_{\text{mmKEM}} \leftarrow \text{mmKEM.GenParam}(1^\lambda, \text{aux})$ 
3:   return  $\text{pp} := (\text{pp}_{\text{mmKEM}})$ 
4: procedure CompilerKEMtoPKE.KeyGen( $\text{pp}$ )
5:   return  $\text{mmKEM.KeyGen}(\text{pp}_{\text{mmKEM}})$ 
6: procedure CompilerKEMtoPKE.Enc( $\text{pp}, (\text{pk}_i)_{i \in [T]}, (m_i)_{i \in [T]}; r_0, r_i$ )
7:    $\text{ct}_0 \leftarrow \text{mmKEM.Encaps}^{\text{ind}}(\text{pp}_{\text{mmKEM}}; r_0)$ 
8:    $\text{ct}_i, k_i \leftarrow \text{mmKEM.Encaps}^{\text{dep}}(\text{pp}_{\text{mmKEM}}; r_0, r_i)$ 
9:    $d_i \leftarrow \text{DEM.Enc}(k_i, m_i)$ 
10:  return  $\text{ct} := (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_T, d_1, \dots, d_T)$ 
11: procedure CompilerKEMtoPKE.Dec( $\text{pp}, \text{sk}, \text{ct}, i$ )
12:    $k \leftarrow \text{mmKEM.Decaps}(\text{pp}_{\text{mmKEM}}, (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_T), i)$ 
13:  return  $\text{DEM.Dec}(d_i, k)$ 

```

Theorem 6.5. *If mmKEM is a key-private mmKEM construction, and DEM is a secure DEM construction, then mmPKE is a key-private mmPKE construction.*

The proof is straightforward and essentially identical to [46].

CCA security. The constructions above only provide CPA security, which is already sufficient for many applications (e.g., see Section 9). However, for completeness, we discuss CCA security as well. One may consider using Fujisaki-Okamoto transformation [23] as prior works [28, 26, 2], where the recipients obtain the randomness used to generate the mKEM ciphertext, verifying that the ciphertext is indeed honestly generated. However, this does not work for us since mmKEM allows a different encapsulated key per recipient: recipients obtaining the randomness can then recover the other recipients' encapsulated keys. Similarly, all other solutions requiring the recipient to learn the randomness do not work (e.g., via trapdoors [44, 43] or other ways [14]).

A natural solution is instead to use zk-SNARKs [25, 12] to guarantee that the ciphertexts are indeed honestly generated. However, the practicality of this solution is also limited. Thus, we do not discuss it in detail and leave it for future works to see how to extend our construction to achieve CCA security efficiently.

Parameter Regime	n	N	q	$\mathcal{D}_\beta$	σ for χ	p	Attack Complexity
1	$\text{poly}(\lambda)$	$O(1)$	Any	$\text{poly}(\lambda)$	$\text{poly}(\lambda), \ll q$	$\leq q$	$(\mathcal{D}_\beta + 2 \cdot \chi)^N$
				Any			$n \cdot (\mathcal{D}_\beta + 2)^N$ [49]
2	$\text{poly}(\lambda)$	$\text{poly}(\lambda)$	$\text{poly}(\lambda)$ and $\mathcal{R}_q$ fully splits	Any	$\ll q$	$\leq q$	$(\chi)^{2N}$
3	$\text{poly}(\lambda)$	$1/ \sigma ^N = \text{negl}(\lambda)$	Any	Any	$1/ \sigma ^N = \text{negl}(\lambda)$	$= q$	q
4	$\text{poly}(\lambda)$	$1/ \sigma ^N = \text{negl}(\lambda)$, $1/(2+\beta)^N = \text{negl}(\lambda)$	$\mathcal{R}_q$ does not split at all	$1/(2+\beta)^N = \text{negl}(\lambda)$	$1/ \sigma ^N = \text{negl}(\lambda)$	$\ll q$	$= \text{MLWE}$

Table 2: Parameters we discussed in Section 7. The first two parameter regimes are broken while the third one we prove to be equivalent to standard Module LWE in Theorem 7.2. The last one is the parameters we need for our construction. Based on the neighborhood search attack [49], we revised our parameters with changes highlighted in blue.

7 Cryptanalysis

In this section, we perform cryptanalysis over OMLWE (Theorem 6.1). We show some attacks to certain parameter regimes and show one parameter regime where OMLWE provably holds under standard MLWE. Lastly, we discuss why the parameter regime we are interested in intuitively holds. These parameter regimes are also summarized in Table 2.

7.1 Broken Parameters

Broken parameter regime 1. For any $\lambda > 0$, let $n = \text{poly}(\lambda), N = 1$ (i.e., LWE instead of MLWE; N can be extended to any $O(1)$), then, for any q , any secret distribution $\mathcal{D}_\beta$ with $\beta = \text{poly}(\lambda)$, Gaussian noise distribution χ where $|\chi| = \text{poly}(\lambda) \ll q$, and any $p \leq q$, we can design the following attack for $\text{OMLWE}_{n,N,q,\mathcal{D}_\beta,\chi,p}$. Given a MLWE sample $(A_1, \vec{b}_1 = A_1 \vec{s} + \vec{e}_1)$, let $p' = |\chi|$, the adversary could set $\vec{a}_A = p' \cdot A_1[1]$ (i.e., the first row of A_1), and obtain $c_1 = \langle \vec{a}_A, \vec{s} \rangle + e_A + u$ and $c_2 = H(\lfloor pu/q \rfloor)$. Then, the attacker simply computes $b' \leftarrow c_1 - p' \cdot \vec{b}_1[1] = (\langle \vec{a}_A, \vec{s} \rangle + e_A + u) - (\langle \vec{a}_A, \vec{s} \rangle - p' \cdot \vec{e}[1]) = e_A + p' \cdot \vec{e}[1] + u$. Then, $\mathcal{A}$ iterates all $e'_A, \vec{e}'[1] \in \chi$ and if $H(\lfloor p(b' \leftarrow c_1 - p' \cdot \vec{b}_1[1] - e'_A - p' \cdot \vec{e}'[1])/q \rfloor) = H(\lfloor pu/q \rfloor)$. This allows $\mathcal{A}$ to recover $\vec{e}[1]$ and thereby the entire $\vec{e}$ by repeating this process for other rows of A_1 . This then allows $\mathcal{A}$ to recover the secret, breaking the OMLWE assumption. The runtime of this attack is essentially $O(|\chi|^2)$ (the time to recover the errors). In Remark 7.1 below, we give an alternative attack with time $O(|\mathcal{D}_\beta| \cdot |\chi|)$ which works better for secret key distribution $\mathcal{D}_\beta$ with a smaller β for completeness.

Remark 7.1 (An alternative attack for parameter regime 1). We provide an alternative attack for broken parameter regime 1 as introduced in Section 7 (with a minor difference that $|\mathcal{D}_\beta| = \text{poly}(\lambda)$). Specifically, any $\lambda > 0$, let $n = \text{poly}(\lambda), N = 1$ (i.e., LWE instead of MLWE; N can be extended to any $O(1)$), then, for any q , any secret distribution $\mathcal{D}_\beta$ with $\beta = \text{poly}(\lambda)$, Gaussian noise distribution χ_σ with standard deviation $\sigma = \text{poly}(\lambda) \ll q/2$, and any $p \leq q$, we can design the following attack for $\text{OMLWE}_{n,N,q,\mathcal{D}_\beta,\chi,p}$.

We start by recovering the first secret element $\vec{s}[0]$. Let $p' = |\chi|$ and set $\vec{a} = (p', 0, 0, \dots, 0)$ as the adversarially chosen vector, then the OMLWE sample contains $c \leftarrow \langle \vec{a}, \vec{s} \rangle + u + e = p' \cdot \vec{s}[0] + u + e \in \mathbb{Z}_q$ and $H(\lfloor pu/q \rfloor)$ where $u \xleftarrow{\$} \mathbb{Z}_q$. For all possible values s', e' of $\vec{s}[0]$ and e , respectively, the attacker

checks if $H(\lfloor p(c - p's' - e')/q \rfloor)$ is equal to $H(\lfloor pu/q \rfloor)$. Notice that in this regime both $\vec{s}[0]$ and e can take $\text{poly}(\lambda)$ different values with high probability. After checking all possible s', e' , if there is only one s' that passes the check, then $\vec{s}[0] = s'$. Note that this happens with probability $O(1/|\chi_\sigma|) = 1/\text{poly}(\lambda)$. Since w_2 can be any $\text{poly}(\lambda)$, the attacker can simply set each row of A_A to be equal to $\vec{a}$ and test. This easily extends to recovering all n secret elements, thus breaking the underlying $\text{OMLWE}_{n,N,q,\mathcal{D}_\beta,\chi,p}$.

Note that, however, a potential mitigation to this attack is to restrict the power of the adversary by limiting A_A . For example, one may require that every row of A_A contains at least half non-zero elements. Then, the above attack immediately fails.

Broken parameter regime 2. Note that the attack above works for $N = O(1)$ as well. For $N = \omega(1)$, the attack above does not directly work since $|\chi|^N$ is super-polynomial in λ (given that $|\chi| = \Omega(\lambda)$). However, a similar attack can be used when rings *fully split* and $q = \text{poly}(\lambda)$, i.e., $\mathcal{R}_q = \prod_{i \in [N]} \mathbb{Z}[X]/(p_i(X))$ for degree one polynomials $p_i(X)$ ⁹. For simplicity, let us assume $n = 1$ (i.e., Ring-LWE instead of Module-LWE). Specifically, the attack works as follows, let $F : \mathcal{R}_q \rightarrow \mathbb{Z}_q^N$ denote the canonical map such that $F(y) = (y \bmod p_1(X), \dots, y \bmod p_N(X))$. Now, the adversary can set $a_A \in \mathcal{R}_q$ such that $F(a_A) = (1, 0, 0, \dots, 0)$. Then, the OMLWE sample returns $b_A \leftarrow a_A s + e_A + u_A = s_0 + e_A + u_A$, $H(\lfloor p \cdot u_A / q \rfloor)$, where $s_0 = F(s)[0]$. In this case, $s_0 \in \mathbb{Z}_q$, and the adversary can iterate all $i \in [s_0]$ and obtains i' such that $F(i') = (i, 0, \dots, 0)$ and then tests whether $H(\lfloor p \cdot (b_A - i') / q \rfloor) = H(\lfloor p \cdot u_A / q \rfloor)$ (which can again be repeated multiple times to remove the rounding difference brought by the error term). This can be used to determine s_0 and, similarly, one can determine all s_i to recover s . This attack no longer works if q is super-polynomial since testing takes $\Omega(q)$ time. More importantly, this attack no longer works for rings that do not split at all. Below, we show that a parameter regime is provably secure (assuming standard MLWE), which brings additional intuition supporting that our assumption likely holds for non-splitting rings, with more details in Section 7.3.

7.2 Provably Secure Parameters

Now we show a parameter regime that is provably secure (under standard MLWE), which is when $1/|\chi| = \text{negl}(\lambda)$ (which essentially requires σ^N being super-polynomial, where σ is the standard deviation of the Gaussian sampling for χ)¹⁰ and when $p = q$.

Lemma 7.2. *For Gaussian distribution χ_σ such that $1/\sigma^N = \text{negl}(\lambda)$ and $p = q$, it holds that $\text{MLWE}_{n,N,q,\mathcal{D},\chi_\sigma} \leq \text{OMLWE}_{n,N,q,\mathcal{D},\chi,p,H}$.*

Proof. We prove the claim by a hybrid argument. For simplicity, we start with $n = 1$ (i.e., Ring-LWE). We argue this via hybrids for any $w_1, w_2, w_A = \text{poly}(\lambda)$:

hyb₀: a simplified OMLWE assumption (see below for why we simplify it) where the adversary is given

$$\begin{aligned} & (a_{1,j}, b_j)_{j \in [w_1]}, \\ & (a_{2,k}, a_{2,k}s + e_{2,k})_{k \in [w_2]}, \\ & (a_{\mathcal{A},i}, c_i := a_{\mathcal{A},i} \cdot s + e_{\mathcal{A},i} + u_{\mathcal{A},i}, H(u_{\mathcal{A},i}))_{i \in [w_A]} \end{aligned}$$

⁹This attack also works for $p_i(X)$ with degree d where $q^d = \text{poly}(\lambda)$.

¹⁰Note that the proof extends to non-Gaussian noise, but for simplicity, we focus on Gaussian noise.

hyb_1 : instead of giving $H(\vec{u}_{\mathcal{A}}[i])$, the adversary is given $\gamma_i \xleftarrow{\$} \mathcal{K}$ (the output space of H)

$$\begin{aligned} & (a_{1,j}, b_j)_{j \in [w_1]}, \\ & (a_{2,k}, a_{2,k}s + e_{2,k})_{k \in [w_2]}, \\ & (a_{\mathcal{A},i}, c_i := a_{\mathcal{A},i} \cdot s + e_{\mathcal{A},i} + u_{\mathcal{A},i}, \gamma_i)_{i \in [w_{\mathcal{A}}]} \end{aligned}$$

For both hybrids, b_j is either a valid RLWE sample or a uniformly random ring element. Now, it is first easy to see that if the adversary can break the assumption in hyb_0 , the adversary can break OMLWE, since $(a_{1,j}, b_j)_{j \in [w_1]}$ can be used to generate $(a_{1,j}, b_j + u_j, H(u_j))_{j \in [w_1]}$ for our original OMLWE sample. The simplification makes the argument below more straightforward.

It is also easy to see that hyb_1 is at least as hard as standard Ring-LWE since γ_i is sampled independently from $u_{\mathcal{A},i}$, and thus $(a_{\mathcal{A},i}, c_i := a_{\mathcal{A},i} \cdot s + e_{\mathcal{A},i} + u_{\mathcal{A},i}, \gamma_i)$ are simply identical to uniformly random samples independent of s .

Thus, the only thing left to prove is that hyb_0 and hyb_1 are indistinguishable. We claim that the adversary can only distinguish the two with $O(1/|\chi|)$ probability where χ is the Gaussian distribution to generate each $e_{\mathcal{A},i}$. More formally, for any PPT adversary $\mathcal{A}$ that either returns 1 or 0, let $\Pr[\text{hyb}_i^{\mathcal{A}}]$ denote the probability that $\mathcal{A}$ returns 1 when in hyb_i for $i \in [0,1]$, then it holds that $\Pr[\text{hyb}_0] - \Pr[\text{hyb}_1] = O(1/|\chi|)$.

Let $\mathcal{E}$ be the event that $\mathcal{A}$ queries the random oracle $H(u_{\mathcal{A},i})$ for any $i \in [w_{\mathcal{A}}]$. It holds that $\Pr[\text{hyb}_1] = \Pr[\text{hyb}_1|\mathcal{E}] \cdot \Pr[\mathcal{E}] + \Pr[\text{hyb}_1|\sim\mathcal{E}] \cdot \Pr[\sim\mathcal{E}]$. Then, if $\sim\mathcal{E}$, no adversary can distinguish hyb_0 from hyb_1 by the definition of a random oracle, i.e., $\Pr[\text{hyb}_1|\sim\mathcal{E}] = \Pr[\text{hyb}_0]$.

Then,

$$\begin{aligned} \Pr[\mathcal{E}] &= \Pr[\mathcal{A} \text{ queries } u_{\mathcal{A},i} \text{ for some } i \in [w_{\mathcal{A}}]] \\ &= \Pr[\mathcal{A} \text{ queries } c_i - a_{\mathcal{A},i} \cdot s - e_{\mathcal{A},i} \text{ for some } i \in [w_{\mathcal{A}}]] \\ &= \Pr[\mathcal{A} \text{ guesses } a_{\mathcal{A},i} \cdot s + e_{\mathcal{A},i}] \leq \Pr[\mathcal{A} \text{ guesses } e_{\mathcal{A},i}] \leq O(1/|\chi|) = O(1/\sigma^N) \end{aligned}$$

Thus, since we have $1/\sigma^N = \text{negl}(\lambda)$, it holds that

$$\begin{aligned} \Pr[\text{hyb}_1] &= \Pr[\text{hyb}_1|\mathcal{E}] \cdot \Pr[\mathcal{E}] + \Pr[\text{hyb}_1|\sim\mathcal{E}] \cdot \Pr[\sim\mathcal{E}] \\ &= \Pr[\text{hyb}_1|\mathcal{E}] \cdot \text{negl}(\lambda) + \Pr[\text{hyb}_0] \cdot (1 - \text{negl}(\lambda)) = \Pr[\text{hyb}_0] + \text{negl}(\lambda) \end{aligned}$$

This proof trivially extends to any $n > 1$ (i.e., Module-LWE than Ring-LWE). $\square$

7.3 Parameters Used in Our mmKEM/mmPKE

In our constructions, we consider the case where $A_{\mathcal{A}}$ contains at least half non-zero elements, $1/|\chi| = \text{negl}(\lambda)$, the ring does not split and $p \ll q$. Observe that this regime avoids all attacks discussed above, but is different than the provable regime since $p \ll q$. For simplicity, we focus on Ring-LWE with $n = 1$.

Now, for $p \ll q$, $\lfloor p \cdot e_{\mathcal{A},i}/q \rfloor = 0$ with high probability, which means that the argument above does not hold anymore. However, let $\mathcal{E}'$ be the event that $\mathcal{A}$ queries the random oracle $H(u')$ where $u' = \lfloor p \cdot u_{\mathcal{A},i}/q \rfloor$ for some $i \in [w_{\mathcal{A}}]$. It still holds that for any PPT adversary $\mathcal{A}$, $\Pr[\text{hyb}_1] = \Pr[\text{hyb}_1|\mathcal{E}'] \cdot \Pr[\mathcal{E}'] + \Pr[\text{hyb}_1|\sim\mathcal{E}'] \cdot \Pr[\sim\mathcal{E}']$.

Then, there are two cases: (1) if we can similarly bound $\Pr[\mathcal{E}'] = \text{negl}(\lambda)$, the rest of the argument still goes through; (2) for the case where $\Pr[\mathcal{E}'] = \text{non-negl}(\lambda)$, we want to show that the adversary learns essentially no additional information.

Intuitively, both seem to be correct for the case where $\mathcal{R}_q$ *does not split* at all. For example, suppose $a_{\mathcal{A},i}(X) = q/p + 0X + \dots + 0X^{N-1}$, it holds that $\lfloor p \cdot a_{\mathcal{A},i}s/q \rfloor$ has super-polynomial number of possibilities. Similarly, for any large enough $a_{\mathcal{A},i}$ (i.e., rounding does not fully remove the information of s), the idea above intuitively holds (i.e., $\Pr[\mathcal{E}'] = \text{negl}(\lambda)$). For smaller $a_{\mathcal{A},i}$ (e.g. $a_{\mathcal{A},i} = 1$) $\Pr[\mathcal{E}']$ can potentially be 1 (or close to 1), since $\lfloor p(a_{\mathcal{A},i}s + e_{\mathcal{A},i} + u_{\mathcal{A},i})/q \rfloor = \lfloor p \cdot u_{\mathcal{A},i}/q \rfloor$ with high probability. However, as shown in [30] (also see a more detailed discussion below), if $a_{\mathcal{A},i}$ is small, then learning $a_{\mathcal{A},i}s + e_{\mathcal{A},i}$ does not break the assumption (the so-called **HintMLWE** assumption), for properly chosen parameters (again, only for non-splitting rings, since this argument requires $|a_{\mathcal{A},i}|$ to be small).

Unfortunately, we are not able to formalize this intuition. Thus, we conjecture that **OMLWE** holds for any non-splitting rings and for any $p \ll q$.

Relationship to HintMLWE. In [30], an MLWE variant called **HintMLWE** is proposed. To summarize, it states that MLWE is secure even if there is a hint, which is the secret key multiplied by a small element being masked by an error. In other words, in addition to regular MLWE samples, the adversary is also given $\gamma_i \cdot \vec{s} + \vec{y}_i$, for $i \in [\ell]$, $\ell = \text{poly}(\lambda)$, where $\gamma_i \xleftarrow{\$} \mathcal{D} \subseteq \mathcal{R}$ (not a vector but a single ring element) and $\vec{y}_i$ is some error vector. It is proven to be equivalent to standard MLWE for small γ_i (up to some parameter loss).

Interestingly, when $n = 1$ (i.e., considering RLWE only), **OMLWE** is a stronger assumption: if one could break **HintMLWE**, one could break **OMLWE**. The reduction is intuitive: using our **mmKEM** schemes (Algorithms 2 and 3), a recipient, who is able to recover the encapsulated keys, can learn the accumulated errors, which are exactly of the form $\gamma_i \cdot \text{sk} + y_i$ with $\text{sk} \in \mathcal{R}$. Thus, the security of our constructions in the RLWE case implies the **HintMLWE** assumption.

This becomes less clear for $n > 1$, since in this case, the hint obtained by the adversary (after removing the encapsulated key) is $\langle \vec{\gamma}_i, \text{sk} \rangle + y_i \in \mathcal{R}$ instead of $\gamma_i \cdot \text{sk} + \vec{y}_i \in \mathcal{R}^n$ as in **HintMLWE**. Thus, it is unclear whether **OMLWE** is stronger than **HintMLWE** in this scenario. We leave the formal discussion of the relationship to future work.

8 Positionless Correctness

The correctness of the **mmKEM** schemes constructed in both Algorithm 2 and Algorithm 3 relies on the fact that recipient i knows its own position index i . This is a relatively strong assumption in practice since it implies coordination between the sender and every intended recipient of every message sent. This seems difficult to achieve when the list of recipients changes often, especially while preserving key privacy (thus one cannot simply attach the public keys and order them lexicographically). Thus in this section, we introduce a generic compiler to enhance any **mmKEM** scheme to a scheme with positionless correctness as defined in Theorem 4.2.

Simple solution. An intuitive solution is to let the recipient try every possible ciphertext. For example, let F be another hash function modeled as a random oracle similar to H , to send $\vec{u}_i$ to recipient i , the sender attaches $F(\vec{u}_i)$ in the corresponding ciphertext ct_i . Then the recipient decapsulates $\vec{u}'_i$ from $(\text{ct}_0, \text{ct}_i)$ for every $i \in [T]$ and checks whether $F(\vec{u}_i) = F(\vec{u}'_i)$. If so, $\vec{u}_i$ is the matched key for recipient i . While this resolves the problem, the runtime of **Decaps** procedure is now $\Omega(T)$.

Polynomial interpolation. As a stepping stone, we replace the hash function by using a polynomial. Then, we are able to improve the linear decapsulation time via polynomial preprocessing

[29] (see Section 3.4).

Let $\mathcal{C}$ be the space of ct_i , where ct_i is the recipient-dependent component for recipient i of the ciphertext (i.e., the output ciphertext of $\text{Encaps}^{\text{dep}}$), and $\mathcal{PK}$ be the space of public key pk . The sender first interpolates a degree $T - 1$ polynomial $f(X) : \mathcal{PK} \rightarrow \mathcal{C}$ that maps a public key to a recipient-dependent ciphertext s.t. $f(\text{pk}_i) = \text{ct}_i$ for all $i \in [T]$, and sends (ct_0, f) as the final ciphertext. Then, the recipient, with public key pk , computes $\text{ct}' \leftarrow f(\text{pk})$ and uses $(\text{ct}_0, \text{ct}')$ to decapsulate the key. Notice that the ciphertext size is slightly reduced compared to the simple solution above.¹¹

Remark 8.1. If the original decapsulation circuit is algebraic, the new circuit with polynomial interpolation remains algebraic (the simple solution requires non-algebraic operations like comparisons). This can be useful in some more involved scenarios. For example, in the case of group oblivious message retrieval [38] the decapsulation needs to be done under FHE, which only efficiently supports algebraic operations like additions and multiplications.

Reducing the runtime. Notice that since the degree of the interpolated function is $T - 1$, the recipient still needs $\Omega(T)$ time to decapsulate its key. To reduce the runtime, instead of interpolating a degree $T - 1$ function, we use an m -variate individual-degree- d -function (i.e., the degree of each variable is at most d), such that $m^d \geq T$ over some finite field $\mathbb{F}_Q$ (Q to-be-determined later).

Formally speaking, the sender interpolates a polynomial $f(X) : \mathbb{F}_Q^m \rightarrow \mathbb{F}_Q$, where $Q \geq |\text{ct}_i|$ is a prime, and $Q^m \geq |\text{pk}|$ (where pk denotes the bit size of $\text{pk} \in \mathcal{PK}$ and $|\text{ct}_i|$ denotes the bit size of $\text{ct}_i \in \mathcal{C}$). For simplicity, we treat $Q \geq |\text{pk}|$ and thus $Q = O(\max(|\text{pk}|, |\text{ct}_i|))$ based on Bertrand's postulate [10]. With $\log(|\text{pk}|) = \text{poly}(\lambda)$ (i.e., the public key from this space has $\text{poly}(\lambda)$ bits¹²) and similarly $|\text{ct}| = \text{poly}(\lambda)$, we can bound $\log(Q)$ to be $\text{poly}(\lambda)$. According to Theorem 3.4, the polynomial interpolation takes $O(m \cdot d^m \cdot \text{polylog}(Q))$ time. Then by employing the polynomial preprocessing technique in Theorem 3.5, every evaluation takes only $\text{poly}(d, m, \log(Q))$ time. As in [35], we set $d = \log^{2/\epsilon}(T)$ for some $\epsilon > 0$ and m such that $d^m \geq T$, which gives us $m = \frac{\epsilon}{2} \log(T) / \log \log(T) + O(1)$.

With these parameters, on the side of the sender, the polynomial interpolation and preprocessing procedure takes $O_\lambda(T^{1+\epsilon})$ time, for any $\epsilon > 0$. On the side of the recipient, the evaluation of the preprocessed f takes $O_\lambda(\text{polylog}(T))$ time¹³. Moreover, since our compiler is generic, future interpolation and preprocessing techniques can be easily embedded to further improve the efficiency.

Hence, the resulting scheme achieves positionless correctness with little overhead asymptotically. The full algorithm is formalized in Algorithm 5.

Theorem 8.2. *If mmKEM is a key private decomposable mmKEM scheme (i.e., with Encaps defined as in Algorithm 1) with regular correctness (Theorem 4.1), then mmKEMcc is a key private mmKEM scheme with positionless correctness (Theorem 4.2). Furthermore, there exists a PolyPreProcess algorithm such that for mmKEMcc, the recipient runtime is $O_\lambda(\text{polylog}(T))$ and the sender runtime is $O_\lambda(T^{1+\epsilon} \cdot \text{polylog}(T))$.*

¹¹W.l.o.g., we assume that $|\text{pk}| \leq |\text{ct}_i|$, where $|\text{pk}|$ denotes the bit size of pk and $|\text{ct}_i|$ denotes the bit size of ct_i . Otherwise, one can use a hash function $K : \mathcal{PK} \rightarrow \{0,1\}^\lambda$ to compress pk before interpolation. Thus, we could represent f with $T \cdot |\text{pk}|$ bits, which is less than $T \cdot |\text{ct}_i|$ in the simple solution.

¹²If $|\text{pk}| = \text{superpoly}(\lambda)$, i.e., there are $\text{superpoly}(\lambda)$ bits per public key, it takes at least $\text{superpoly}(\lambda)$ time to generate it, which breaks the efficiency of the underlying scheme.

¹³Alternatively, we could set $d > 2^{\sqrt{T}}$ and $m = \lceil \log_d T \rceil \leq \sqrt{\log T} + O(1)$. In this case, the preprocessing time is $O_\lambda(T^{1+o(1)})$ and the evaluation time is $O_\lambda(T^{o(1)})$.

Algorithm 5 Compile mmKEM to obtain conditionless correctness mmKEM

```

1: procedure mmKEMcc.GenParam( $1^\lambda, \text{aux}$ )
2:    $\text{pp}_{\text{mmKEM}} \leftarrow \text{mmKEM.GenParam}(1^\lambda, \text{aux})$ 
3:   Find the smallest prime  $Q$  such that  $Q \geq |\mathcal{PK}|$  and  $Q \geq |\mathcal{C}|$ 
4:   Set some constant  $\epsilon > 0$ .
5:   return  $\text{pp} = (\text{pp}_{\text{mmKEM}}, Q, \epsilon)$ 
6: procedure mmKEMcc.KeyGen( $\text{pp}$ )
7:    $(\text{pk}_{\text{mmKEM}}, \text{sk}_{\text{mmKEM}}) \leftarrow \text{mmKEM.KeyGen}(\text{pp}_{\text{mmKEM}})$ 
8:   return  $\text{pk} := \text{pp}_{\text{mmKEM}}, \text{sk} := (\text{pk}_{\text{mmKEM}}, \text{sk}_{\text{mmKEM}})$ 
9: procedure mmKEMcc.Encaps( $\text{pp}, \vec{\text{pk}} = (\text{pk}_1, \dots, \text{pk}_n); r_0, r_1, \dots, r_T$ )
10:   $\text{ct}_0 \leftarrow \text{mmKEM.Encaps}^{\text{ind}}(\text{pp}_{\text{mmKEM}}; r_0)$ 
11:  for  $i \in [T]$  do
12:     $(\text{ct}_i, k_i) \leftarrow \text{mmKEM.Encaps}^{\text{dep}}(\text{pp}_{\text{mmKEM}}, \text{pk}_i; r_0, r_i)$ 
13:    Set  $m, d$  such that  $m^d \geq T$ , and  $m$  be the smallest integer such that  $m > \log^{2/\epsilon}(T)$ .
14:    Interpolate function  $f : \mathbb{Z}_Q^n \rightarrow \mathbb{Z}_Q$  such that  $f(\text{pk}_i, \text{pk}_i, \dots, \text{pk}_i) = \text{ct}_i$  for all  $i \in [T]$  (note
      that since  $Q \geq \max(|\mathcal{PK}|, |\mathcal{C}|)$ , we have a simple one to one mapping from  $\text{pk}$  or  $\text{ct}$  to  $\mathbb{Z}_Q$ ).
15:     $\tilde{f} \leftarrow \text{PolyPreProcess}(f)$ 
16:    return  $(\text{ct} := (\text{ct}_0, \tilde{f}), (k_i)_{i \in [T]})$ 
17: procedure mmKEMcc.Decaps( $\text{pp}, \text{sk}, \text{ct}, i = \perp, \text{pk}$ )
18:  Evaluate the preprocessed function  $\text{ct}' \leftarrow \tilde{f}(\text{pk}, \text{pk}, \dots, \text{pk})$ 
19:   $k \leftarrow \text{mmPKE.Decaps}(\text{pp}_{\text{mmPKE}}, \text{sk}_{\text{mmKEM}}, (\text{ct}_0, \text{ct}'), i = 1)$ 
20:  return  $k$ 

```

Proof. • (Positionless correctness) For function f , it holds that $\text{ct}' := f(\text{pk}_i) = \text{ct}_i$ for recipient i . Then, by the correctness of mmKEM, $\text{mmKEM.Decaps}(\text{ct} = (\text{ct}_0, \text{ct}'), 1)$ decapsulates the correct key.

- (Semantic security) Note that all procedures in Algorithm 5 use mmKEM as a black box without touching the encapsulated keys. Thus the semantic security (either CPA or CCA security) trivially holds.
- (Key privacy) Let $\text{ct} := (\text{ct}_0, \text{ct}_1, \dots, \text{ct}_T)$ be the ciphertext generated for $\vec{\text{pk}}_0$ and $\text{ct}' := (\text{ct}'_0, \text{ct}'_1, \dots, \text{ct}'_T)$ be the ciphertext generated for $\vec{\text{pk}}_1$ where $\vec{\text{pk}}_0$ and $\vec{\text{pk}}_1$ are vectors of public keys chosen by the adversary as in Fig. 1. Then, since mmKEM is key-private, it holds that the distribution of ct and ct' are computationally indistinguishable. Therefore, for *any* vector of points, $(v_1, \dots, v_T)$, let f be the function such that $f(v_i) = \text{ct}_i$ and f' be the function such that $f'(v_i) = \text{ct}'_i$, it holds that the distribution of f and f' are computationally indistinguishable trivially. Therefore, it holds that mmKEMcc is also key-private.
- (Efficiency analysis) As analyzed above, when using the PolyPreProcess in Theorem 3.5, for mmKEMcc.Encaps, the interpolation and the preprocessing take $O_\lambda(T^{1+\epsilon})$, so the overall time for encapsulation is $O_\lambda(T^{1+\epsilon} + T \cdot T_{\text{mmKEM.Encaps}^{\text{dep}}} + T_{\text{mmKEM.Encaps}^{\text{ind}}})$. Since mmKEM.Encaps takes $O_\lambda(T)$ time, which also calls mmKEM.Encaps^{dep} for T times and mmKEM.Encaps^{ind} for one time (as in Algorithm 1), the overall runtime for mmKEMcc.Encaps is thus $O(T^{1+\epsilon})$.

For mmKEMcc.Decaps , the evaluation of $\tilde{f}$ takes $O_\lambda(\text{polylog}(T))$ time, so the total time for decapsulation is $O_\lambda(\text{polylog}(T) + T_{\text{mmKEM.Decaps}})$ where $T_{\text{mmKEM.Decaps}}$ is the runtime of mmKEM.Decaps when for a single recipient, which is $O_\lambda(1)$. Therefore, the runtime of mmKEMcc.Decaps is $O_\lambda(\text{polylog}(T))$. $\square$

Remark 8.3. The compiler in Algorithm 5 for mmKEM can be extended to mmPKE in the most straightforward way: simply replace the function interpolation over the mmKEM ciphertexts with interpolation over the mmPKE ciphertexts. The performance analysis remains the same.

Also note that if we let PolyPreProcess do nothing (i.e., return the input polynomial), we are back to the solution discussed in Remark 8.1. For small T , this would be preferable in practice.

9 Applications

In this section, we show some applications that only require CPA-security for $\text{mmKEM}/\text{mmPKE}$, which our scheme can practically achieve as shown above.

9.1 Application to Batched Random Oblivious Transfer

The state-of-the-art post-quantum secure random oblivious transfer (rOT) introduced in [41] uses Crystals-kyber as a building block. At a high-level, in rOT the sender holds two random strings x_0, x_1 , the receiver holds a bit b and the rOT outputs x_b to the receiver. The scheme in [41] based on a CPA-secure KEM works as follows: the receiver generates a public key pk (of a KEM scheme) and a random string r . Then, it sets $\text{pk}_b = \text{pk}$ and $\text{pk}_{1-b} = r - \text{pk}$ and sends both to the sender. The sender encrypts x_i under pk_i and sends back to the receiver. The receiver decrypts to obtain x_b , without learning anything about x_{1-b} , while the sender learns nothing about b . The KEM scheme used in this construction is Crystals-Kyber.

However, in many applications, batched rOT (e.g., [32]) instead of a single-instance rOT is used. Instead of having two strings, the sender has λ pairs of strings, and the recipient select one of them for each of the pairs. Thus, the above construction is repeated λ times.

Applying our mmKEM . One could alternatively use mmKEM to allow the sender to send λ random strings with one ciphertext, instead of generating λ ciphertexts independently, to improve the efficiency. However, using an mmKEM *without* malicious security breaks the privacy requirement of rOT, since the recipient could maliciously craft the public keys to learn both x_b and x_{1-b} (e.g., by recovering the randomness used by the sender with the attacks we introduced in Section 5). Hence, applying mmKEM to batched rOT requires malicious security. Concretely, with $\lambda = 128$, our ciphertext size is reduced by $> 11\times$ as shown in Section 10. Thus, using mmKEM , instead of naively using Crystals-Kyber for λ times, could greatly improve the communication efficiency in the second round¹⁴.

9.2 Application to Group Oblivious Message Retrieval

Another application of mmKEM (mmPKE) is Group Oblivious Message Retrieval (GOMR) (an extension of the primitive Oblivious Message Retrieval [37, 39, 34, 36, 27, 19]). Essentially, (ad-

¹⁴Note that in the first round, the recipient still needs to send the public keys. However, in applications where the first round public keys can be reused, this cost can be amortized.

	$n \cdot N$	q	N	σ	h	p	r
Param1	896	2^{19}	128	2	150	2^{10}	2^8
Param2	896	2^{19}	64	2	150	2^{10}	2^8
Param3	1752	2^{58}	8	2^{16}	280	2^{16}	2^{42}

Table 3: Parameters for our mmKEM.

hoc) GOMR lets the sender arbitrarily choose T recipients, send them a message, and put the message on the bulletin board. Each of the recipients uses its key to find the messages that are addressed to them¹⁵. In [38], the authors achieve this by letting the sender attach T independent ciphertexts (encrypting zeros) of a CPA-key-private PKE scheme to the message, each of which is encrypted under one intended recipient’s public key to indicate that recipient (obliviously). These T ciphertexts together are called a *clue*. The recipient can use its secret key to decrypt the ciphertext and a ciphertext decrypted into zero indicates that the message is for that recipient.

It is easy to see that instead of using T independent ciphertexts, we can use a single CPA-key-private mmPKE ciphertext. This reduces the clue size from $O(T \cdot |\text{ct}|)$ into $O(T + |\text{ct}|)$ asymptotically. Concretely, a saving similar to Fig. 3 can be achieved, which will be about $10.5\times$ smaller for $T = 100$.

In [38], the authors discussed using mmPKE for ad-hoc GOMR, but they observe that the existing lattice-based schemes do not have security against maliciously-generated public keys. Thus, our construction solves this problem.

10 Evaluation

In this section, we analyze the parameter selection and evaluate the efficiency of our scheme in comparison to existing solutions (where the size estimation is via calculation and runtime estimation is via microbenchmarks detailed below). Moreover, we benchmark the runtime of our mmKEM scheme in Algorithm 3 assuming polynomial modulus-to-noise ratio.

Note that in an older version, we proposed certain parameters that are concretely insecure due to the attack in [49]. See Appendix A for details.

Parameter selection. For our mmKEM scheme (Algorithm 3), we choose the Ring-LWE parameters to be: $n \cdot N = 896$, $q = 2^{19}$, $N = 128$, $\sigma = 2$, let the secret keys be binary vectors with hamming weight $h = 150$, and $p = 2^{10}$, $r = 2^8$. Under these parameters, we achieve a computational security of > 128 bits [20] (plus that the attacks summarized in Table 2 also require $\gg 2^{128}$ of computation cost) and correctness $> 1 - 2^{-128}$. We denote this parameter set as **Param1**.

Note that $N = 128$ is chosen mainly to mitigate the attack in [49], and we can alternatively choose a slightly more aggressive parameter: everything else being the same as above except that $N = 64$, and the attack in [49] requires at least 2^{114} steps (even ignoring the hidden constants). We denote this parameter set as **Param2**.

Lastly, we choose a third parameter set: $n \cdot N = 1752$, $q = 2^{58}$, $N = 8$, $\sigma = 2^{16}$, and let the secret keys be binary vectors with hamming weight $h = 280$, and $p = 2^{16}$, $r = 2^{42}$. Note that one difference is that we choose the randomness sampled by the sender to be uniform from $[0, 2^{16}]$

¹⁵GOMR in fact lets the recipients outsource this job to a third-party, but for the case of this application, we simplify the model for illustration.

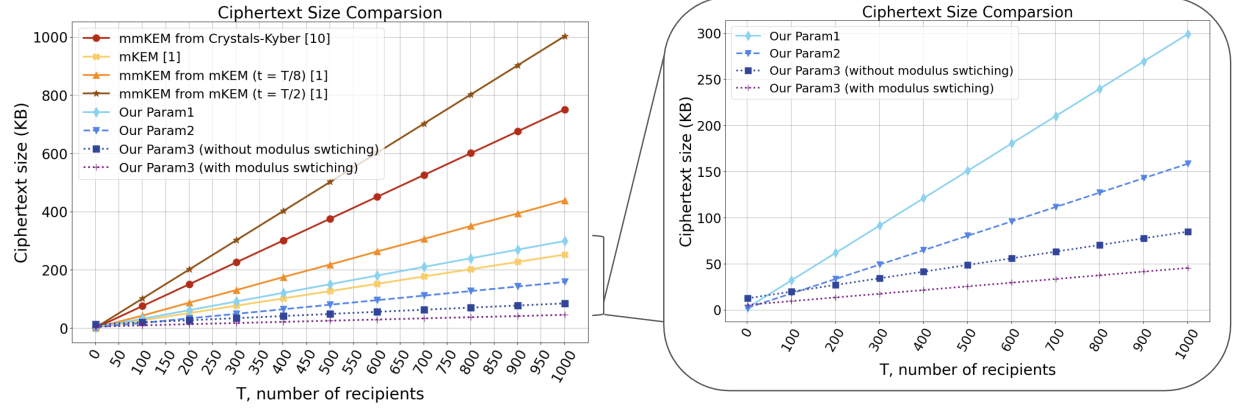


Figure 3: Ciphertext size comparisons with prior works. t means the number of different plaintext keys for T different recipients for [2].

instead of being binary. This guarantees 128-bit security against the attack in [49] as well (and also 128-bit security by [20] and other attacks in Table 2). We denote this parameter set as **Param3**.

All three sets of parameters are summarized in Table 3.

Additional optimizations. There are two additional optimizations we employed for our scheme for evaluation. The first is modulus switching: essentially, the ciphertext after encryption has a large fraction of error. Thus, one technique reduce the ciphertext size by removing these errors is modulus switching [15]. For a ciphertext $ct \in \mathbb{Z}_q^*$, one can compute $\lfloor q'ct/q \rfloor$ for $q' \ll q$. This introduces additional noise. For our purpose, since we want all the noise to be all positive, instead of regular rounding, we perform a $\lfloor q'ct/q \rfloor$, such that the noise produced by modulus switching is positive. Note that this noise is bounded by the hamming weight of the secret key plus one (i.e., $h + 1$), since the rounding/flooring for a single element is bounded by 1. Note that for the first two parameters, modulus switching does not help much since σ is already small. However, it helps greatly for **Param3** where we can choose $q' = 2^{25}$. which both still guarantee a correctness of 128-bit.¹⁶ The second possible optimization is to use RLWE instead of MLWE as mentioned in Remark 6.4.¹⁷ We report our performance evaluations both with and without these optimizations.

Ciphertext size compared to other mmKEMs. Since there exist no prior works on lattice-based multi-message multi-recipient KEM, to demonstrate our practicality, we compare our schemes with (1) the naive solution by directly using Crystals-Kyber [13] to generate an individual ciphertext for each recipient (denoted as mmKEM rom Crystals-Kyber in Fig. 3), and (2) the state-of-the-art multi-recipient KEM solution [2]. For the latter, we compare our result with their mKEM with three different sets of parameters: (1) their mKEM (i.e., every recipient shares the same key); (2) mmKEM from their mKEM for $t = T/8$ (i.e., every 8 consecutive recipients share the same key); (3) mmKEM from their mKEM for $t = T/2$ (i.e., every 2 consecutive recipients share the same

¹⁶Note that with modulus switching error, the off-by-one error in Section 6.3 can be off-by-two. Thus, our testing algorithm needs to be changed accordingly. However, this is straightforward: simply test more possible keys. We report the expected runtime difference below.

¹⁷Note that this optimization changes the underlying assumption as mentioned in Remark 6.4 but our cryptanalysis in Section 7 still applies the same way.

key).¹⁸ Note that as mentioned in Section 2, their functionality and the privacy guarantee are both weaker than ours.

From Fig. 3, we see that for 1000 recipients, the ciphertext size of our scheme can be as small as $> 16\times$ smaller than the trivial solution of using Crystals-Kyber. For a smaller group like 100 recipients, our ciphertext size is also about $2-8\times$ smaller than the trivial solution. And even compared to the mMKE in [2], our scheme is still about $2-6\times$ smaller (for 50-1000 recipients). Furthermore, it is easy to see that when recipients require different keys, the construction in [2] behaves worse (and becomes worse than the trivial solution for $T/2$ different keys).

Other efficiency metrics. The public key sizes for our two parameter sets (Param1, Param2) are 2.1KB, and Param3 is 12.4KB, when using the optimization in Remark 6.4. They are 2716.5KB, 14.5KB respectively without optimization. As a comparison, the public key size of Crystals-Kyber is 0.78 KB. The larger public key size (for the optimized version) for our mMKE scheme is to make sure that the ratio q/r is large enough so that the key correction procedure demonstrated in Section 6.3 could be done efficiently. The unoptimized version serves more similarly to LWE (since it has a small ring dimension) and thus has a large public key size.

We also estimate the runtime for correcting the errors, where we use SHA256 in OpenSSL [48]. We collect microbenchmarks using GCP e2-standard-4 instance with 4 virtual CPU and 16GB RAM. Our key encapsulation runtime for the two-parameter sets Param1, Param2 are dominated by SHA256 tests: requiring 2^{22} and 2^{19} tests (for 128-bit correctness), for ~ 0.1 ms per 1000 tests. For Param3, only 2^8 tests are needed. When using modulus switching, Param3 instead requires 3^8 tests.

Extending to mmPKE. We can make similar conclusions to all above for our mmPKE scheme (using the generic KEM-DEM paradigm as shown in Section 6.4) compared to the naive solution and prior works [28, 26, 2], since the underlying mMKE (or mMKE) is the major bottleneck for performance, as also shown in [26].

Acknowledgements

We are grateful to anonymous reviewers for insightful suggestions, and especially for pointing out broken parameter regime 2 in Section 7.1 (i.e., when a ring fully splits over $q = \text{poly}(\lambda)$). We also thank Yibin Yang for discussing the application of OT, and thank Markku-Juhani Saarinen for noticing the confusion of the evaluation section. We thank Hongxiao Wang for discussing the neighborhood search attack in [49] and for proposing an alternative mitigation method for our Param3 (see Appendix A for details). Zeyu Liu is supported by Yale CADMY.

¹⁸Recall that they introduced mmPKE from their mMKE, and this mMKE from mMKE is simply their mmPKE with random plaintexts.

References

- [1] S. Agrawal, E. Kirshanova, D. Stehlé, and A. Yadav. Practical, round-optimal lattice-based blind signatures. In H. Yin, A. Stavrou, C. Cremers, and E. Shi, editors, *ACM CCS 2022*, pages 39–53, Los Angeles, CA, USA, Nov. 7–11, 2022. ACM Press.
- [2] J. Alwen, D. Hartmann, E. Kiltz, M. Mularczyk, and P. Schwabe. Post-quantum multi-recipient public key encryption. Cryptology ePrint Archive, Report 2022/1046, 2022. <https://eprint.iacr.org/2022/1046>.
- [3] G. Asharov, A. Jain, A. López-Alt, E. Tromer, V. Vaikuntanathan, and D. Wichs. Multiparty computation with low communication, computation and interaction via threshold FHE. In D. Pointcheval and T. Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 483–501, Cambridge, UK, Apr. 15–19, 2012. Springer, Heidelberg, Germany.
- [4] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon. The Messaging Layer Security (MLS) Protocol. RFC 9420, July 2023.
- [5] M. Bellare, A. Boldyreva, A. Desai, and D. Pointcheval. Key-privacy in public-key encryption. In *ASIACRYPT 2001*, LNCS, pages 566–582. Springer, Heidelberg, Germany, Dec. 9–13, 2001.
- [6] M. Bellare, A. Boldyreva, K. Kurosawa, and J. Staddon. Multirecipient encryption schemes: How to save on bandwidth and computation without sacrificing security. *IEEE Transactions on Information Theory*, 53(11):3927–3943, 2007.
- [7] M. Bellare, A. Boldyreva, and J. Staddon. Randomness re-use in multi-recipient encryption schemes. In Y. G. Desmedt, editor, *Public Key Cryptography — PKC 2003*. Springer Berlin Heidelberg, 2002.
- [8] M. Bellare, C. Namprempe, D. Pointcheval, and M. Semanko. The one-more-RSA-inversion problems and the security of Chaum’s blind signature scheme. *Journal of Cryptology*, 16(3):185–215, June 2003.
- [9] L. Benz, W. Beskorovajnov, S. Eilebrecht, R. Gröll, M. Müller, and J. Müller-Quade. Chosen-ciphertext secure dual-receiver encryption in the standard model based on post-quantum assumptions. Cryptology ePrint Archive, Paper 2024/094, 2024.
- [10] J. Bertrand. *Mémoire sur le nombre de valeurs que peut prendre une fonction: quand on y permute les lettres qu’elle renferme*. Bachelier, 1845.
- [11] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In S. Goldwasser, editor, *ITCS 2012*, pages 326–349, Cambridge, MA, USA, Jan. 8–10, 2012. ACM.
- [12] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In S. Goldwasser, editor, *ITCS 2012*, pages 326–349, Cambridge, MA, USA, Jan. 8–10, 2012. ACM.
- [13] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehle. Crystals - kyber: A cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroSecP)*, pages 353–367, 2018.

- [14] X. Boyen, M. Izabachène, and Q. Li. A simple and efficient CCA-secure lattice KEM in the standard model. In C. Galdi and V. Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 321–337, Amalfi, Italy, Sept. 14–16, 2020. Springer, Heidelberg, Germany.
- [15] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In *ITCS 2012*, Jan. 8–10, 2012.
- [16] D. R. L. Brown and R. P. Gallant. The static diffie-hellman problem, 2004. Submitted to Eurocrypt 2005 (preliminary version) dbrown@certicom.com 12958 received 15 Nov 2004, last revised 24 Jun 2005.
- [17] S. S. M. Chow, M. K. Franklin, and H. Zhang. Practical dual-receiver encryption - soundness, complete non-malleability, and applications. In J. Benaloh, editor, *CT-RSA 2014*, volume 8366 of *LNCS*, pages 85–105, San Francisco, CA, USA, Feb. 25–28, 2014. Springer, Heidelberg, Germany.
- [18] M. Christ and S. Gunn. Pseudorandom error-correcting codes. CRYPTO 2024, 2024.
- [19] H. Chu, X. Wang, and Y. Jia. Private signaling secure against actively corrupted servers. Cryptology ePrint Archive, Paper 2025/1056, 2025.
- [20] B. Curtis, C. Lefebvre, F. Virdia, F. Göpfert, J. Owen, L. Ducas, M. Schmidt, M. Albrecht, R. Player, and S. Scott. Security estimates for the learning with errors problem.
- [21] T. Diament, H. K. Lee, A. D. Keromytis, and M. Yung. The dual receiver cryptosystem and its applications. In V. Atluri, B. Pfitzmann, and P. McDaniel, editors, *ACM CCS 2004*, pages 330–343, Washington, DC, USA, Oct. 25–29, 2004. ACM Press.
- [22] Module-lattice-based key-encapsulation mechanism standard. <https://csrc.nist.gov/pubs/fips/203/ipd>, Aug. 2024.
- [23] E. Fujisaki and T. Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, Jan. 2013.
- [24] C. Gentry, C. Peikert, and V. Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In C. Dwork, editor, *Proceedings of the 40th Annual ACM Symposium on Theory of Computing, Victoria, British Columbia, Canada, May 17-20, 2008*, pages 197–206. ACM, 2008.
- [25] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- [26] K. Hashimoto, S. Katsumata, E. Postlethwaite, T. Prest, and B. Westerbaan. A concrete treatment of efficient continuous group key agreement via multi-recipient PKEs. In G. Vigna and E. Shi, editors, *ACM CCS 2021*, pages 1441–1462, Virtual Event, Republic of Korea, Nov. 15–19, 2021. ACM Press.
- [27] Y. Jia, V. Madathil, and A. Kate. HomeRun: High-efficiency oblivious message retrieval, unrestricted. CCS 2024, 2024.

- [28] S. Katsumata, K. Kiwata, F. Pintore, and T. Prest. Scalable ciphertext compression techniques for post-quantum KEMs and their applications. In S. Moriai and H. Wang, editors, *ASIACRYPT 2020, Part I*, volume 12491 of *LNCS*, pages 289–320, Daejeon, South Korea, Dec. 7–11, 2020. Springer, Heidelberg, Germany.
- [29] K. S. Kedlaya and C. Umans. Fast polynomial factorization and modular composition. *SIAM Journal on Computing*, 40(6):1767–1802, 2011.
- [30] D. Kim, D. Lee, J. Seo, and Y. Song. Toward practical lattice-based proof of knowledge from hint-MLWE. In H. Handschuh and A. Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 549–580, Santa Barbara, CA, USA, Aug. 20–24, 2023. Springer, Heidelberg, Germany.
- [31] D. Kim, D. Lee, J. Seo, and Y. Song. Toward practical lattice-based proof of knowledge from hint-mlwe. In H. Handschuh and A. Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, pages 549–580, Cham, 2023. Springer Nature Switzerland.
- [32] V. Kolesnikov, R. Kumaresan, M. Rosulek, and N. Trieu. Efficient batched oblivious PRF with applications to private set intersection. In E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, editors, *ACM CCS 2016*, pages 818–829, Vienna, Austria, Oct. 24–28, 2016. ACM Press.
- [33] K. Kurosawa. Multi-recipient public-key encryption with shortened ciphertext. In D. Naccache and P. Paillier, editors, *Public Key Cryptography*. Springer Berlin Heidelberg.
- [34] K. Lee and Y. Yeo. SophOMR: Improved oblivious message retrieval from SIMD-aware homomorphic compression. Cryptology ePrint Archive, Report 2024/1814, 2024.
- [35] W.-K. Lin, E. Mook, and D. Wichs. Doubly efficient private information retrieval and fully homomorphic RAM computation from ring LWE. Cryptology ePrint Archive, Paper 2022/1703, 2022.
- [36] Z. Liu, K. Sotiraki, E. Tromer, and Y. Wang. Snake-eye resistant PKE from LWE for oblivious message retrieval and robust encryption. In *EUROCRYPT 2025, Part III*, LNCS, pages 126–156, June 2025.
- [37] Z. Liu and E. Tromer. Oblivious message retrieval. In Y. Dodis and T. Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022*, pages 753–783, Cham, 2022. Springer Nature Switzerland. Full version: Cryptology ePrint Archive 2021; internal citations follow the latter’s numbering.
- [38] Z. Liu, E. Tromer, and Y. Wang. Group oblivious message retrieval. *Cryptology ePrint Archive*, 2023.
- [39] Z. Liu, E. Tromer, and Y. Wang. PerfOMR: Oblivious message retrieval with reduced communication and computation. In *USENIX Security 2024*. USENIX Association, Aug. 2024.
- [40] V. Lyubashevsky, C. Peikert, and O. Regev. On ideal lattices and learning with errors over rings. *J. ACM*, 2013.

- [41] D. Masny and P. Rindal. Endemic oblivious transfer. In L. Cavallaro, J. Kinder, X. Wang, and J. Katz, editors, *ACM CCS 2019*, pages 309–326, London, UK, Nov. 11–15, 2019. ACM Press.
- [42] D. Micciancio and C. Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In D. Pointcheval and T. Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 700–718, Cambridge, UK, Apr. 15–19, 2012. Springer, Heidelberg, Germany.
- [43] D. Micciancio and C. Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In D. Pointcheval and T. Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 700–718, Cambridge, UK, Apr. 15–19, 2012. Springer, Heidelberg, Germany.
- [44] C. Peikert and B. Waters. Lossy trapdoor functions and their applications. *SIAM Journal on Computing*, 40(6):1803–1844, 2011.
- [45] A. Pinto, B. Poettering, and J. C. Schuldt. Multi-recipient encryption, revisited. In *Proceedings of the 9th ACM Symposium on Information, Computer and Communications Security*, ASIA CCS ’14, New York, NY, USA, 2014. Association for Computing Machinery.
- [46] A. Pinto, B. Poettering, and J. C. N. Schuldt. Multi-recipient encryption, revisited. In S. Moriai, T. Jaeger, and K. Sakurai, editors, *ASIACCS 14*, pages 229–238, Kyoto, Japan, June 3–6, 2014. ACM Press.
- [47] V. Shoup. A proposal for an ISO standard for public key encryption. Cryptology ePrint Archive, Paper 2001/112, 2001.
- [48] The OpenSSL Project. OpenSSL: Cryptography and ssl/tls toolkit. <https://www.openssl.org/>, 1998-2025. An ongoing project, first released in 1998.
- [49] H. Wang, M. F. Esgin, R. Steinfeld, M.-J. O. Saarinen, and S.-M. Yiu. A practical neighborhood search attack on oracle MLWE. Cryptology ePrint Archive, Paper 2026/177, 2026.
- [50] H. Wang, R. Steinfeld, M.-J. O. Saarinen, M. F. Esgin, and S.-M. Yiu. mmCipher: Batching post-quantum public key encryption made bandwidth-optimal. USENIX Security 2026, 2026.
- [51] Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. <https://github.com/zama-ai/tfhe-rs>.
- [52] D. Zhang, K. Zhang, B. Li, X. Lu, H. Xue, and J. Li. Lattice-based dual receiver encryption and more. In W. Susilo and G. Yang, editors, *ACISP 18*, volume 10946 of *LNCS*, pages 520–538, Wollongong, NSW, Australia, July 11–13, 2018. Springer, Heidelberg, Germany.

A A Recent Attack on OMLWE

A recent work identifies a neighborhood search attack [49] on our Oracle MLWE (OMLWE) assumption (Theorem 6.1), and shows that the concrete parameter choices in Appendix B are insecure, and we have updated our parameters in Section 10. The attack is extended from the discussion in a concurrent and independent work on lattice-based mmKEM/mmPKE [50]. We recall the attack algorithm at the end of this section, after assessing its ramifications for our scheme.

Attack efficiency. In short, for ring dimension N , attack of [49] breaks OMLWE in $O(N \cdot (\|\vec{s}\|_\infty + 2)^N)$ time (for $\ell = N$), where s is the short secret. It thus implies a new parameter regime in which OMLWE is broken, namely when $(\|\vec{s}\|_\infty + 2)^N = \text{poly}(\lambda)$, where $\vec{s}$ is our underlying secret key. Specifically, the two concrete parameter sets instantiated in Section 10 *do not* provide 128-bit security as originally expected. Concretely, they show that these two parameter sets can be broken in under one minute.

This means that our schemes in Algorithms 2 and 3 are broken for our parameter choices in Section 10. However, they remain plausibly secure for larger parameters (discussed below), since the attack requires $\exp(N)$ time while our schemes run in only $\tilde{O}(N)$ time (even for polynomial modulus with the key-testing technique, as argued in Section 6.3).

Revised parameters. To resist the attack, we revise our parameters to $N = 128$, $n \cdot N = 896$, $q = 2^{18}$, $\sigma = 1$, with a binary secret key with fixed Hamming weight $h = 150$, $p = 2^{10}$, and $r = 230$. Under these parameters, the runtime complexity exceeds 2^{128} for all known attacks, namely [49], Section 7, and the attacks considered in LWE estimator [20].

These parameters do lead to a larger ciphertext size (approximately 300 KB for 1000 recipients) compared to the two parameter sets presented in Section 10. Nevertheless, this remains roughly $\sim 2.5\times$ smaller than a naive deployment of Crystals-Kyber when the number of recipients is sufficiently large.

In terms of decryption time, the dominant cost remains the error correction procedure described in Section 6.3. To guarantee a decryption failure probability below 2^{-128} , approximately 2^{22} tests are required under these parameters. For a failure probability of 2^{-80} , only 2^{15} tests are sufficient.

Alternatively, we may choose $N = 64$ while keeping all other parameters unchanged. This parameter set is more aggressive; however, for a binary key, the newly proposed attack requires more than 2^{116} steps to break the assumption (ignoring the hidden constants), and it attains ≥ 124 -bit security under the LWE estimator [20] and our own cryptanalysis. With this parameter set, the ciphertext size becomes ~ 160 KB for 1000 recipients, approximately $\sim 4.7\times$ smaller than the naive use of Crystals-Kyber for sufficiently large T . The decryption requires 2^{19} and 2^{12} tests for 2^{-128} and 2^{-80} failure probability respectively.

An alternative mitigation. An alternative way to mitigate the attack is to increase the norm of $\vec{s}$. Importantly, $\vec{s}$ here refers to the randomness used by the encapsulation algorithms (i.e., lines 15 and 20 in Algorithm 3), instead of the secret key, which therefore allows us to use modulus switching to further reduce the ciphertext size. Specifically, we can choose $\vec{s}$ uniformly from 2^{16} , which then allows us to choose $n \cdot N = 1752$, $q = 2^{58}$, $N = 8$, $\sigma = 2^{16}$, and let the secret keys be binary vectors with hamming weight $h = 280$, and $p = 2^{16}$, $r = 2^{42}$. The decryption requires only 2^8 tests and 3^8 tests, without and with modulus switching, respectively. The ciphertext size is only 85 and 45KB, respectively, for 1000 recipients.

We updated our Section 10 to reflect these changes and keep our old evaluation section in Appendix B.

Further discussion on security. Although the proposed parameter sets above are not directly broken by the newly proposed attack [49], our OMLWE assumption is newly introduced and therefore still warrants further study. This new result [49], together with our cryptanalysis in Section 7, provides important initial steps and serves as an insightful foundation for future work. We believe that the OMLWE assumption itself, in addition to allowing the construction of mmPKE/mmKEM, may be of independent interest.

Concurrent work. As mentioned, a concurrent and independent work [50] provides an elegant and practical alternative solution for lattice-based mmKEM/mmPKE. In short, they use non-interactive zero-knowledge proofs (NIZK) to prove that public keys are honestly generated for the construction discussed in Section 5 (with MLWE instead of LWE). Since the public keys are honestly generated, the attacks of Section 5 no longer apply. They prove the construction secure under hint MLWE [31], which is known to be equivalent to LWE for the required parameters. Importantly, the proof requires MLWE since even honestly generated public keys may reveal additional hints, which is properly addressed in [50].

As benchmarked in [50], this alternative solution offers better performance than our constructions in terms of ciphertext size, at the cost of appending an NIZK proof to the public key, which incurs a relatively small overhead as shown in [50, Table 4].

Thus, for practical deployment of mmKEM/mmPKE, the solution in [50] may be preferred. However, our techniques provide an alternative approach that does not rely on NIZK and introduce a new, expressive assumption, OMLWE, both of which may be of independent interest.

Neighborhood search attack. For completeness, we recall the neighborhood search of [49].

At a high level, this attack exploits a structural property of the concrete instantiations of OMLWE, namely the relatively small ring dimension. This feature enables an exhaustive neighborhood search over the bounded error space (which becomes infeasible once the ring dimension is moderately large).

Concretely, given the challenge matrices A_1, A_2 , the adversary defines the vector $\vec{s}' := (1, 0, \dots, 0) \in \mathcal{R}^{w_1}$ and $\vec{e}'_i := (0, \dots, \underset{i\text{-th element}}{\gamma}, \dots, 0)$, where $\gamma \geq \lceil q/p \rceil + 2 \cdot |\chi|$. The adversary then constructs $A_{\mathcal{A}}$,

corresponding to a maliciously chosen public key in mmPKE, as $\begin{pmatrix} \vec{b}_1^\top \\ \vdots \\ \vec{b}_\ell^\top \end{pmatrix}$, with $\vec{b}_i = A_1 \vec{s}' + \vec{e}'_i \in \mathcal{R}^n$.

Given $\vec{v} = A_{\mathcal{A}} \vec{s} + \vec{e}_{\mathcal{A}} + \vec{u}_{\mathcal{A}}$ and $A_1 \vec{s} + \vec{e}_1$ in the OMLWE instance, let $\vec{u}'_{\mathcal{A}}[i] = \lfloor p \cdot \vec{u}_{\mathcal{A}}[i] / q \rfloor$ and $k_i = \vec{u}_{\mathcal{A}}[i] - \lfloor q \cdot \vec{u}'_{\mathcal{A}}[i] / p \rfloor$. The adversary computes

$$\begin{aligned} c_i &= \vec{v}[i] - \langle A_1 \vec{s} + \vec{e}_1, \vec{s}' \rangle \\ &= (A_1 \vec{s}' + \vec{e}'_i)^\top \vec{s} + \vec{e}_{\mathcal{A}}[i] + \vec{u}_{\mathcal{A}}[i] - (A_1 \vec{s} + \vec{e}_1)^\top \vec{s}' \\ &= \vec{e}'_i{}^\top \vec{s} - \vec{e}_1[1] + \vec{e}_{\mathcal{A}}[i] + \vec{u}_{\mathcal{A}}[i] \\ &= \gamma \cdot \vec{s}[i] - \vec{e}_1[1] + \vec{e}_{\mathcal{A}}[i] + \lfloor q \cdot \vec{u}'_{\mathcal{A}}[i] / p \rfloor + k_i. \end{aligned}$$

As argued in [49], the error term for c_i satisfies

$$\|\vec{e}'_i\| := \|\gamma \cdot \vec{s}[i] - \vec{e}_1[1] + \vec{e}_{\mathcal{A}}[i] + k_i\|_\infty < \lceil q/p \rceil (\|\vec{s}\|_\infty + 2),$$

where $\mathcal{D}$ denotes the distribution of the secret key $\vec{s}$. Therefore,

$$\lfloor p \cdot c_i / q \rfloor = \lfloor p \cdot \vec{e}'_i / q \rfloor + \lfloor p \cdot \vec{u}_{\mathcal{A}}[i] / q \rfloor.$$

	$n \cdot N$	q	N	σ	h	p	r
Param1	768	2^{25}	16	2^8	280	2^8	2^{17}
Param2	1024	2^{41}	8	2^{16}	350	2^{16}	2^{25}

Table 4: Parameters for our mmKEM with 128-bit security and correctness.

With a bounded norm on e'_i , the adversary launches the neighborhood search attack by brute-forcing all $(\|\vec{s}\|_\infty + 2)^N$ possible values of e'_i , where N is the ring dimension, and checks whether $H(\lfloor p \cdot c_i/q \rfloor)$ matches the hash value $H(\lfloor p \cdot \vec{u}_A[i]/q \rfloor)$ provided in the OMLWE instance.

Once the correct e'_i is identified, the adversary recovers the secret coefficient via

$$\vec{s}[i] := \lfloor (c_i - \lfloor q \cdot \vec{u}_A[i]/p \rfloor) \gamma \rfloor.$$

Repeating this procedure for all coefficients, the overall runtime of the adversary is $O(\ell \cdot (\|\vec{s}\|_\infty + 2)^N)$, as stated in [49, Corollary 4].

B Evaluation

In this section, we analyze the parameter selection and evaluate the efficiency of our scheme in comparison to existing solutions (where the size estimation is via calculation and runtime estimation is via microbenchmarks detailed below). Moreover, we benchmark the runtime of our mmKEM scheme in Algorithm 3 assuming polynomial modulus-to-noise ratio.

Parameter selection. For our mmKEM scheme (Algorithm 3), we choose the Ring-LWE parameters to be: $n \cdot N = 768$, $q = 2^{25}$, $N = 16$, $\sigma = 2^8$, let the secret keys be binary vectors with hamming weight $h = 280$, and $p = 2^8$, $r = 2^{17}$. Under these parameters, we achieve a computational security of > 128 bits [20] (plus that the attacks we propose in Section 7 also require $\gg 2^{128}$ of computation cost) and correctness $> 1 - 2^{-128}$. We denote this parameter set as **Param1**.

We also choose another parameter set: $n \cdot N = 1024$, $q = 2^{41}$, $N = 8$, $\sigma = 2^{16}$, let the secret keys be binary vectors with hamming weight $h = 350$, and $p = 2^{16}$, $r = 2^{25}$. Similarly, this guarantees 128-bit security and correctness. We denote this parameter set as **Param2**. Both sets of parameters are summarized in Table 4.

Additional optimizations. There are two additional optimizations we employed for our scheme for evaluation. The first is modulus switching: essentially, the ciphertext after encryption has a large fraction of error. Thus, one technique reduce the ciphertext size by removing these errors is modulus switching [15]. For a ciphertext $\text{ct} \in \mathbb{Z}_q^*$, one can compute $\lfloor q' \text{ct}/q \rfloor$ for $q' \ll q$. This introduces additional noise. For our purpose, since we want all the noise to be all positive, instead of regular rounding, we perform a $\lfloor q' \text{ct}/q \rfloor$, such that the noise produced by modulus switching is positive. Note that this noise is bounded by the hamming weight of the secret key plus one (i.e., $h + 1$), since the rounding/flooring for a single element is bounded by 1. Thus, for **Param1**, we choose $q' = 2^{19}$ and for **Param2**, we choose $q' = 2^{25}$, which both still guarantee a correctness of 128-bit.¹⁹ The second possible optimization is to use RLWE instead of MLWE as mentioned in Remark 6.4.²⁰ We report our performance evaluations both with and without these optimizations.

Ciphertext size compared to other mmKEMs. Since there exist no prior works on lattice-based multi-message multi-recipient KEM, to demonstrate our practicality, we compare our schemes

¹⁹Note that with modulus switching error, the off-by-one error in Section 6.3 can be off-by-two. Thus, our testing algorithm needs to be changed accordingly. However, this is straightforward: simply test more possible keys. We

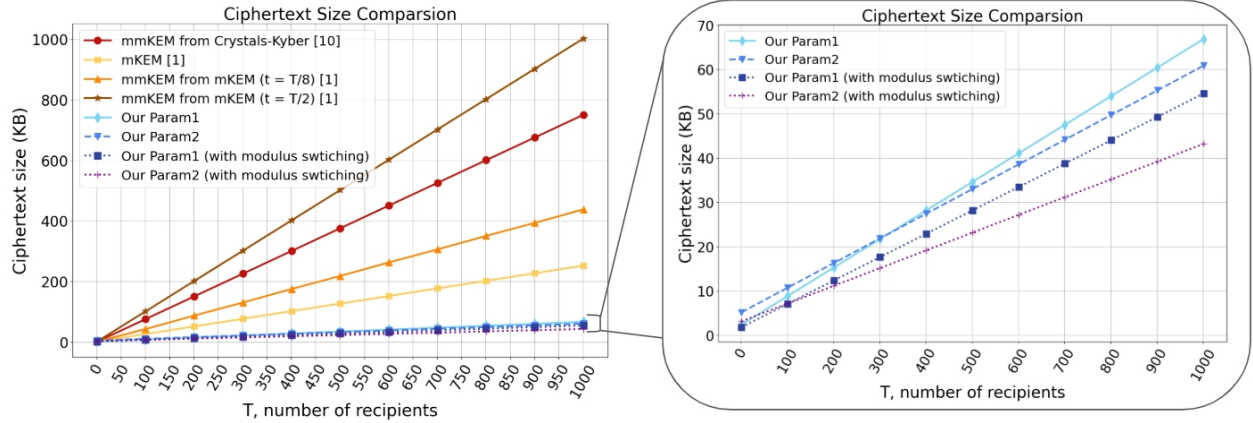


Figure 4: Ciphertext size comparisons with prior works. t means the number of different plaintext keys for T different recipients for [2].

with (1) the naive solution by directly using Crystals-Kyber [13] to generate an individual ciphertext for each recipient (denoted as mmKEM from Crystals-Kyber in Fig. 4), and (2) the state-of-the-art multi-recipient KEM solution [2]. For the latter, we compare our result with their mKEM with three different sets of parameters: (1) their mKEM (i.e., every recipient shares the same key); (2) mmKEM from their mKEM for $t = T/8$ (i.e., every 8 consecutive recipients share the same key); (3) mmKEM from their mKEM for $t = T/2$ (i.e., every 2 consecutive recipients share the same key).²¹ Note that as mentioned in Section 2, their functionality and the privacy guarantee are both weaker than ours.

From Fig. 4, we see that for 1000 recipients, the ciphertext size of our scheme is almost $20\times$ smaller than the trivial solution of using Crystals-Kyber. For a smaller group like 50 recipients, our ciphertext size is also about $5\text{--}10\times$ smaller than the trivial solution. And even compared to the mKEM in [2], our scheme is still about $4\text{--}10\times$ smaller (for 50-1000 recipients). Furthermore, it is easy to see that when recipients require different keys, the construction in [2] behaves worse (and becomes worse than the trivial solution for $T/2$ different keys).

Other efficiency metrics. The public key sizes for our two parameter sets (Param1, Param2) are 5.1KB, 2.3KB respectively, when using the optimization in Remark 6.4. They are 656KB, 112KB respectively without optimization. As a comparison, the public key size of Crystals-Kyber is 0.78 KB. The larger public key size (for the optimized version) for our mmKEM scheme is to make sure that the ratio q/r is large enough so that the key correction procedure demonstrated in Section 6.3 could be done efficiently. The unoptimized version serves more similarly to LWE (has a small ring dimension) and thus has a large public key size.

For our micro-benchmarks (used to estimate the runtime of our construction), we use TFHE-rs [51] as the base library, which is the state-of-the-art implementation of power-of-two Ring-

report the expected runtime difference below.

²⁰Note that this optimization changes the underlying assumption as mentioned in Remark 6.4 but our cryptanalysis in Section 7 still applies the same way.

²¹Recall that they introduced mmPKE from their mKEM, and this mmKEM from mKEM is simply their mmPKE with random plaintexts.

LWE/Module-LWE encryption, and OpenSSL [48] for SHA256. We collect microbenchmarks using GCP e2-standard-4 instance with 4 virtual CPU and 16GB RAM. Our key encapsulation runtime for the two-parameter sets **Param1**, **Param2** are about $(3 + 3T)$ ms, $(5 + 5T)$ ms respectively. With $T = 1000$, our key decapsulation runtime is about 10ms and 5ms respectively, without using modulus switching: it takes 2^{16} SHA256 to test whether a key is correct for **Param1** (~ 0.1 ms per 1000 tests) which takes about 3ms and takes 2^8 SHA256 for **Param2** which cost negligible extra time. When using modulus switching, in the *worst case*, the decapsulation time becomes about 5s and 5.5ms, since now for **Param1** it takes 3^{16} tests while for **Param2** there are still only 3^8 tests. Note that, however, this is the worst-case scenario: in practice (average case), it still takes $\ll 10$ ms to decapsulate for **Param1** since on average, it takes much fewer tests.

Extending to mmPKE. We can make similar conclusions to all above for our mmPKE scheme (using the generic KEM-DEM paradigm as shown in Section 6.4) compared to the naive solution and prior works [28, 26, 2], since the underlying mmKEM (or mKEM) is the major bottleneck for performance, as also shown in [26].